

Spring Reactor

Ali Koudri – ali.koudri@janus-academy.fr



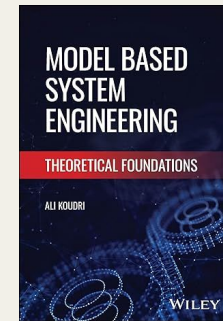
Votre formateur

Ali Koudri

- PhD en informatique
- 25+ ans en ingénierie système et logicielle, modélisation, performance, sécurité
- IRT SystemX - Thales - CEA
- Janus Academy : formation & conseil indépendant
- Approche cross-stack : front, back, ci/cd, observabilité

Publications

Model Based Systems Engineering: Theoretical Foundations

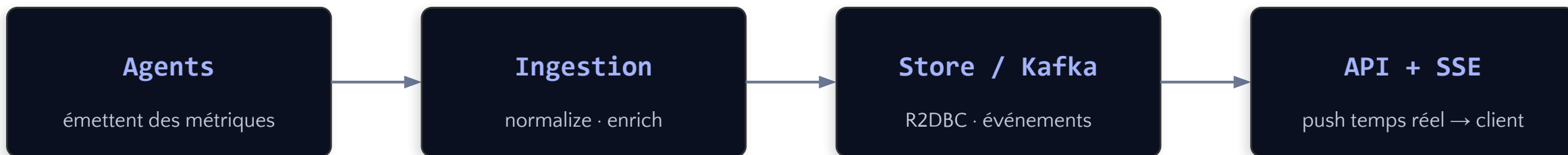


Posture pédagogique

Démarche systémique, mesure avant optimisation, capitalisation transposable

Pulse — agrégateur de télémétrie

Pulse collecte les métriques émises par des agents, les normalise et les enrichit à la volée, les persiste, applique des règles d'alerte, et diffuse l'état en temps réel. C'est un système de supervision orienté flux.



Capacités

- Ingestion en flux : normalisation et enrichissement asynchrone des échantillons.
- Persistance réactive (R2DBC / PostgreSQL) et ingestion d'événements (Kafka).
- Règles d'alerte et agrégation de santé par fan-out vers des services tiers.
- Diffusion temps réel vers le client (SSE / WebSocket) avec backpressure.

Clôner le repo suivant : <https://github.com/akoudri/pulse-reactor.git>

Architecture & stack

PULSE-COMMON

DTOs & contrats d'API partagés (records).

PULSE-REACTIVE

Le service réactif : WebFlux, R2DBC, Reactor.

PULSE-MVC-LOOM

Le jumeau : Spring MVC + virtual threads.

PULSE-LOADTEST

Charge & mesure différentielle (Gatling).

DEUX JUMEAUX, UN CONTRAT

pulse-reactive (non bloquant, WebFlux/R2DBC) et pulse-mvc-loom (MVC + virtual threads) exposent les mêmes endpoints et DTOs — pour comparer réactif et Loom sur des mesures.

Stack — Java 21+ · Spring Boot 4 · WebFlux / Reactor 2025.0 · R2DBC + PostgreSQL · Spring Kafka · Gatling.

Spring Reactor

Partie 1 — Fondations & maîtrise des flux



Les six étapes du parcours

01

Pourquoi le réactif

I/O non bloquant • Loom

02

Modèle conceptuel

Streams • souscription

03

Créer des Mono/Flux

Factory • defer • Sinks

04

Transformer

map • flatMap • concatMap

05

Gérer les erreurs

resume • retry • timeout

06

Tester les flux

StepVerifier • temps virtuel

Fil rouge — Pulse, un agrégateur de télémétrie : chaque concept s'incarne dans le pipeline d'ingestion que vous construisez en lab.

01

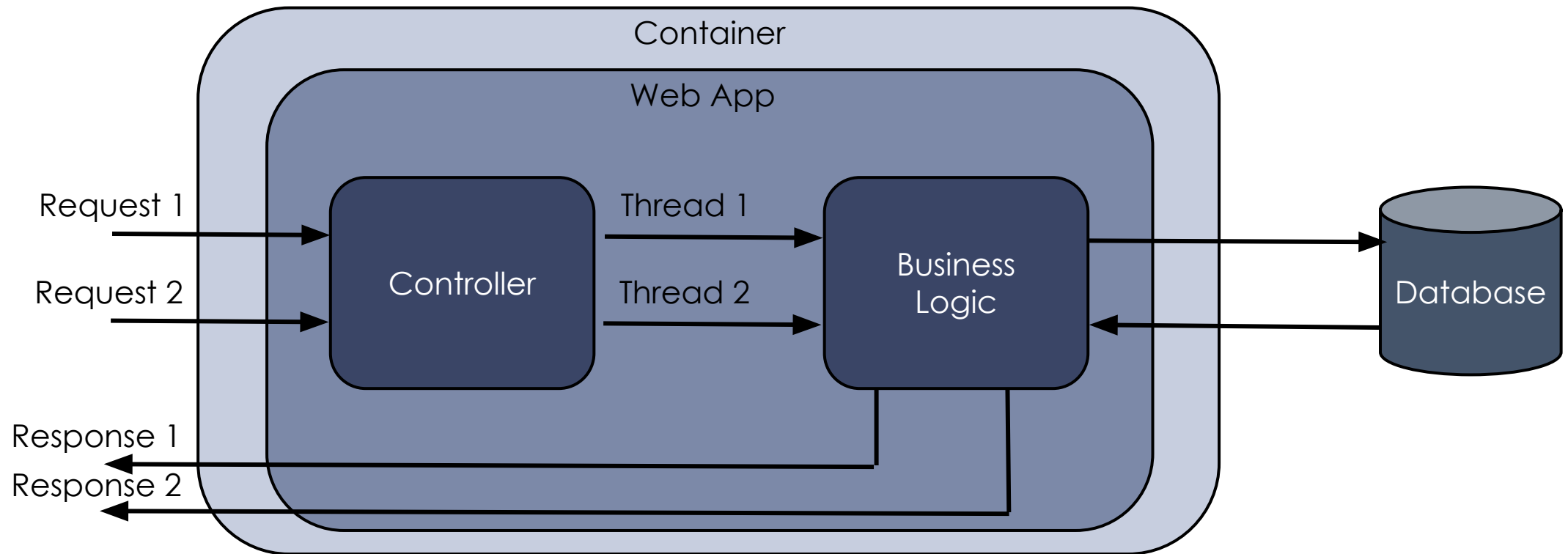
Pourquoi le réactif?

I/O non bloquant, élasticité — et la question Loom posée d'emblée.



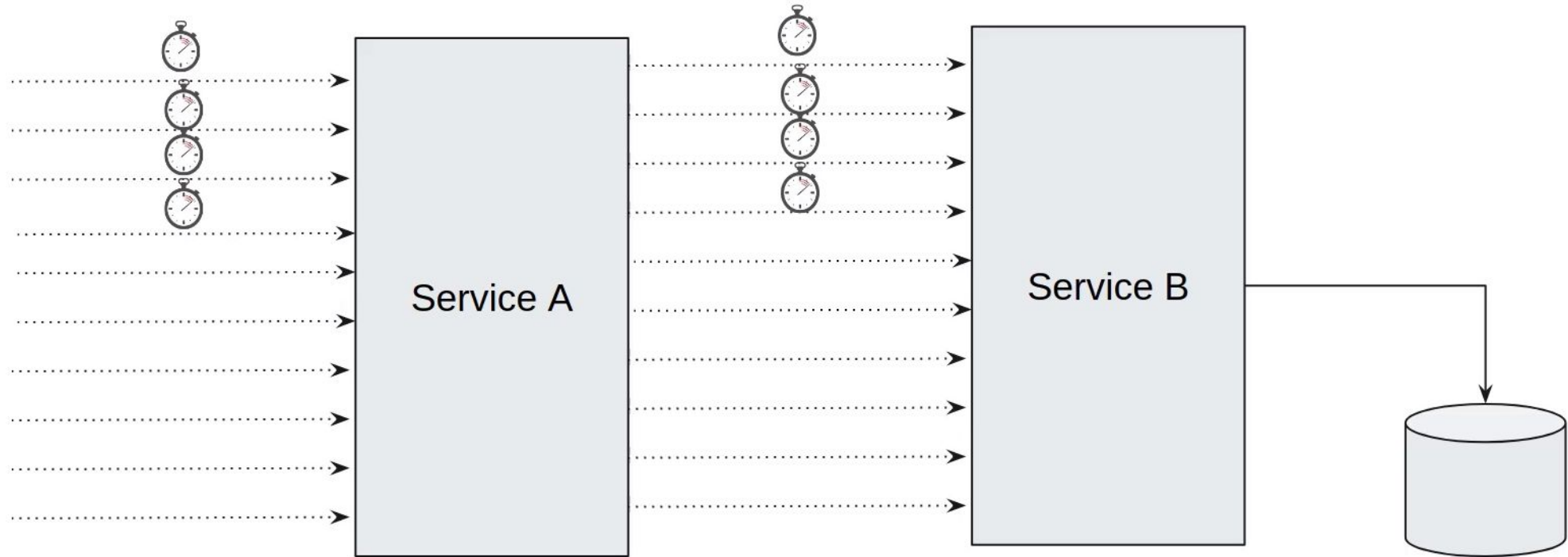
Analyse d'une requête HTTP

- Chaque requête crée un thread bloquant
- Le serveur est configuré avec un pool de threads limité



Composition de services

- Approche gourmande en ressources et peu scalable !



Bloquant vs non bloquant

- Un thread par requête qui attend une I/O = un thread immobilisé, coûteux en mémoire.
- Sous forte concurrence I/O-bound, on sature le pool de threads avant le CPU.
- Le réactif libère le thread pendant l'attente : peu de threads, beaucoup de connexions.
- Le prix : un modèle de composition asynchrone, déclaratif

L'IDÉE CLÉ

Le réactif n'accélère pas une requête isolée.

Il améliore le débit sous charge en cessant de gaspiller des threads à attendre.

REACTIVE STREAMS

Une spec, plusieurs implémentations.

Reactor est celle de l'écosystème Spring.

« Pourquoi pas juste des virtual threads ? »

- Les virtual threads (Java 21+) rendent le code bloquant impératif quasi gratuit en threads.
- Pour beaucoup de CRUD I/O-bound, MVC + Loom suffit — et reste débogable.
- Mais Loom ne donne ni backpressure, ni composition de flux, ni propagation de changement.
- Le réactif garde l'avantage sur le streaming continu, le SSE/WebSocket, l'event-driven.

CAS DIFFÉRENTIEL — FIL ROUGE

Même endpoint Pulse, deux implémentations :

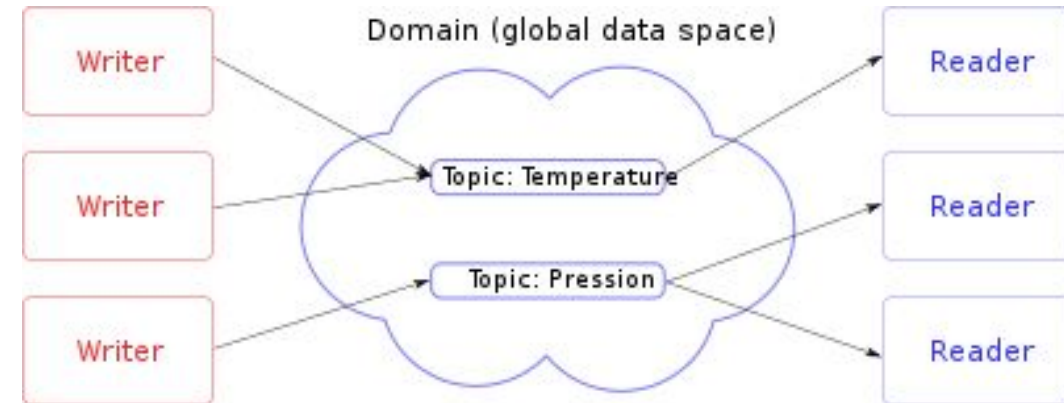
- règles d'alerte (CRUD) → MVC + Loom
- ingestion temps réel → réactif

On mesurera les deux en Partie 4. Les chiffres tranchent, pas le dogme.

« Pourquoi pas juste le pattern Publish / Subscribe ? »

Mécanisme de publication et d'abonnement dans lequel:

- Les diffuseurs (publishers) n'émettent pas des messages à des destinataires (subscribers) particuliers
- Les diffuseurs émettent des messages à des catégories particulières (topics) sans avoir à se soucier si ces derniers seront consommés par des destinataires quelconques
- À l'inverse, les destinataires s'abonnent à des catégories sans soucier de savoir si il y aura des producteurs, ni même qui produira des messages



Limitations du pattern

- Il ne passe pas à l'échelle,
- Il gère mal les erreurs ou les tests: impossible de tester les composants sans avoir à charger tout le contexte
- Les données sont produites quand bien même il n'y a pas de composants pour les consommer

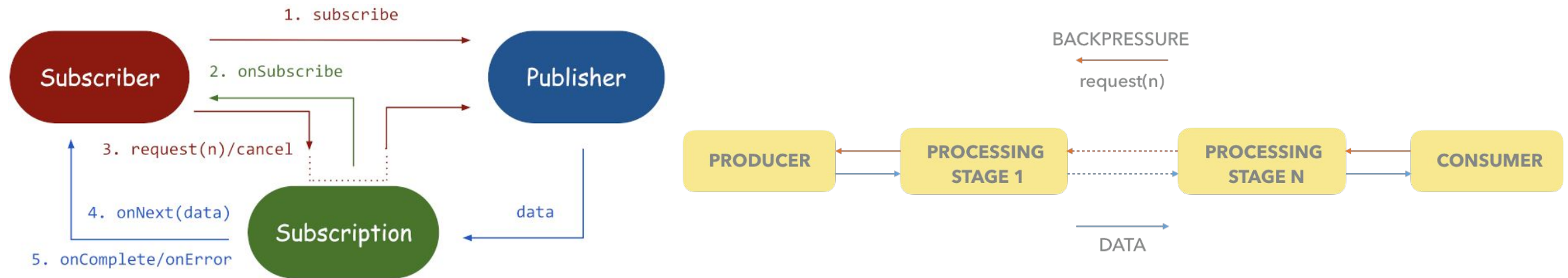
Reactive Streams : le contrat



- Quatre signaux : `onSubscribe`, `onNext*`, et au plus un `onError` | `onComplete`.
- Le Subscriber pilote le rythme via `request(n)` — c'est le backpressure, par construction.
- `push` = `request(MAX)` · `pull` = `request(1)` · `push-pull` = adaptation des rythmes.

Reactive Streams dans Java

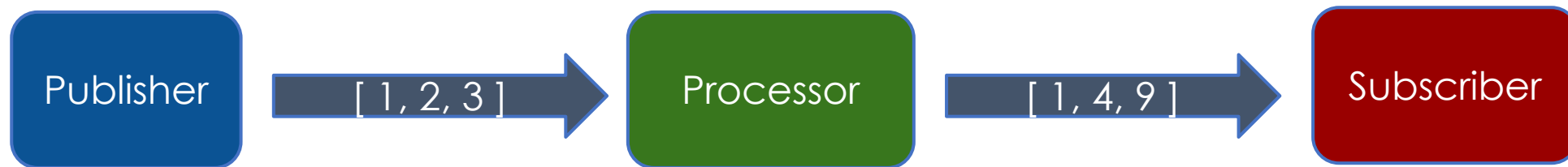
- Initiative visant à établir un standard pour le traitement de flux asynchrones non-bloquants
 - Introduit dans la version 9 de Java
- Interfaces définies dans le paquetage `java.util.concurrent.Flow`
 - Permettent de créer des programmes asynchrones basés sur des séquences observables d'événements
 - Étendent le patron de conception "Observer" en ajoutant un itérateur
 - Évite de produire de la données s'il n'y a pas de consommateur



Reactive Streams dans Java – Exercice

Implanter la chaîne représentée par ce schéma:

- Un publisher publier une série d'entiers
- Un transformer (publisher / subscriber) met ces entiers au carré
- Un subscriber stocke les résultats dans un tableau



Quand le réactif paie, et quand il ne paie pas

LE RÉACTIF GAGNE

- Streaming continu (métriques, événements)
- SSE / WebSocket, temps réel
- Backpressure réel de bout en bout
- Fan-out/fan-in vers de nombreux services
- Pipelines de transformation composés

LOOM SUFFIT SOUVENT

- CRUD I/O-bound classique
- Appels séquentiels à une base / un service
- Logique métier impérative simple
- Équipe sans expérience réactive
- Débogabilité prioritaire

02

Modèle conceptuel

Le contrat de demande, et la règle d'or : rien ne se passe sans souscription.



Histoire de la réactivité

- La programmation réactive est née dans les laboratoires de Microsoft en 2005, sous la direction de Erik Meijer
 - Sur le sujet de la "programmabilité" du Cloud
 - Sur les problématiques liées au déploiement de services "large-scale asynchronous and data-intensive"
- Le travail fourni par l'équipe de Erik Meijer a abouti en 2007 à la première version publique de la librairie Rx.NET
- Cette librairie a ensuite été portée sur différents langages, dont Java avec la librairie RxJava
- Il est intéressant de noter que RxJava est utilisée de manière native par beaucoup d'applications comme Couchbase ou MongoDB

Reactive Manifesto

Selon le "Reactive Manifesto" (<https://www.reactivemanifesto.org/>):

- Les systèmes modernes font face à des contraintes de plus en plus fortes. Pour y répondre, les systèmes doivent être plus robustes, plus résilients, plus flexibles et mieux placés pour répondre aux exigences modernes [...]
- Nous voulons des systèmes réactifs, résilients, élastiques et pilotés par les messages. Nous les appelons systèmes réactifs.
- Les systèmes construits en tant que systèmes réactifs sont plus flexibles, faiblement couplés et évolutifs. Ils sont donc plus faciles à développer et à modifier. Ils sont beaucoup plus tolérants à l'égard des défaillances et, lorsque celles-ci se produisent, ils y font face avec élégance plutôt qu'en catastrophe. Les systèmes réactifs offrent aux utilisateurs un retour d'information interactif efficace.
- Un système réactif est: **responsive, résilient, élastique, message-driven**.

Reactive Manifesto

Le système doit être responsive :

- Le système réagit en temps voulu, dans la mesure du possible. La réactivité est la pierre angulaire de la convivialité et de l'utilité, mais plus encore, la réactivité signifie que les problèmes peuvent être détectés rapidement et traités efficacement. Les systèmes réactifs s'attachent à fournir des temps de réponse rapides et cohérents, en établissant des limites supérieures fiables afin de fournir une qualité de service constante. Ce comportement cohérent simplifie à son tour le traitement des erreurs, renforce la confiance de l'utilisateur final et l'incite à interagir davantage.

Le système doit être résilient :

- Le système reste réactif en cas de défaillance. Cela ne s'applique pas seulement aux systèmes hautement disponibles et critiques – tout système qui n'est pas résilient ne sera pas réactif après une panne. La résilience est obtenue par la réplication, le confinement, l'isolation et la délégation. Les défaillances sont contenues dans chaque composant, isolant les composants les uns des autres et garantissant ainsi que certaines parties du système peuvent tomber en panne et se rétablir sans compromettre le système dans son ensemble. La reprise de chaque composant est déléguée à un autre composant (externe) et la haute disponibilité est assurée par la réplication si nécessaire. Le client d'un composant n'a pas à gérer ses défaillances.

Reactive Manifesto

Le système doit être élastique :

- Le système reste réactif sous une charge de travail variable. Les systèmes réactifs peuvent réagir aux changements du débit d'entrée en augmentant ou en diminuant les ressources allouées pour servir ces entrées. Cela implique des conceptions sans points de contention ni goulots d'étranglement centraux, ce qui permet de partager ou de répliquer les composants et de répartir les entrées entre eux. Les systèmes réactifs prennent en charge les algorithmes de mise à l'échelle prédictifs, ainsi que réactifs, en fournissant des mesures de performance pertinentes en temps réel. Ils assurent l'élasticité de manière rentable sur des plates-formes matérielles et logicielles de base.

Le système doit être "Message-driven" :

- Les systèmes réactifs s'appuient sur le passage de messages asynchrones pour établir une frontière entre les composants qui garantit un couplage faible, l'isolation et la transparence de l'emplacement. Cette frontière fournit également les moyens de déléguer les défaillances sous forme de messages. L'utilisation du passage de messages explicite permet la gestion de la charge, l'élasticité et le contrôle du flux en façonnant et en surveillant les files d'attente de messages dans le système et en appliquant une pression de retour si nécessaire. L'utilisation de la messagerie transparente comme moyen de communication permet à la gestion des pannes de fonctionner avec les mêmes constructions et la même sémantique à travers un cluster ou au sein d'un seul hôte. La communication non bloquante permet aux destinataires de ne consommer des ressources que lorsqu'ils sont actifs, ce qui réduit la surcharge du système.

Reactive Manifesto

Le Reactive Manifesto défini **le back-pressure** comme suit:

Lorsqu'un composant a du mal à suivre, le système dans son ensemble doit réagir de manière raisonnable. Il est inacceptable que le composant sous tension connaisse une défaillance catastrophique ou qu'il abandonne des messages de manière incontrôlée. Puisqu'il ne peut pas faire face et qu'il ne peut pas tomber en panne, il doit communiquer le fait qu'il est sous tension aux composants en amont et leur demander de réduire la charge. Cette contre-pression est un mécanisme de retour d'information important qui permet aux systèmes de répondre gracieusement à la charge plutôt que de s'effondrer sous celle-ci. La contre-pression peut remonter jusqu'à l'utilisateur, auquel cas la réactivité peut se dégrader, mais ce mécanisme garantit la résilience du système en cas de charge et fournit des informations qui peuvent permettre au système lui-même d'appliquer d'autres ressources pour aider à répartir la charge.

Reactor – Présentation

Reactor est né de la nécessité de réaliser des applications logiciels pouvant facilement passer à l'échelle et pouvant gérer de grandes quantités de données

- Dans le cadre du projet Spring XD (<https://docs.spring.io/spring-xd/docs/current-SNAPSHOT/reference/html/>)
- En optimisant l'utilisation des ressources de mémoire et de calcul
- En optimisant les temps de réponse
- En offrant un cadre cohérent aux développeurs

Reactor a été conçu autour d'un pattern comportemental, le "Reactor pattern"

- Événements asynchrones / traitements synchrones
- <https://www.dre.vanderbilt.edu/~schmidt/PDF/reactor-siemens.pdf>

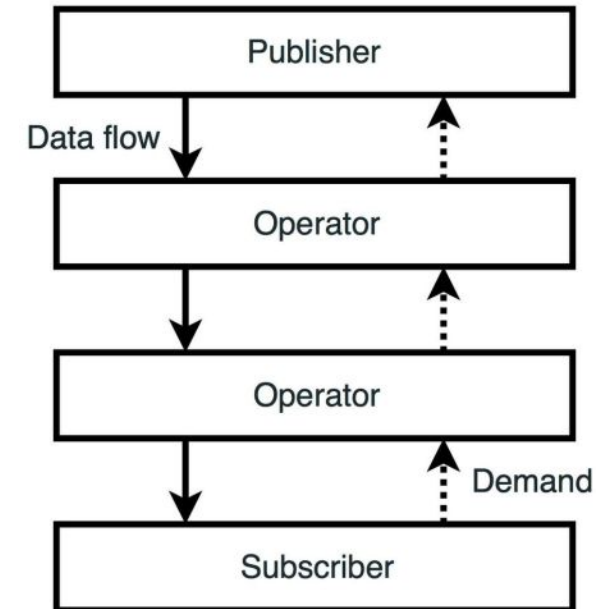
Reactor – Objectifs

Une grande partie de la complexité de la création d'applications Big Data réelles est liée à l'intégration de nombreux systèmes disparates en une solution cohérente pour toute une série de cas d'utilisation. Les cas d'utilisation courants rencontrés lors de la création d'une solution big data complète sont les suivants:

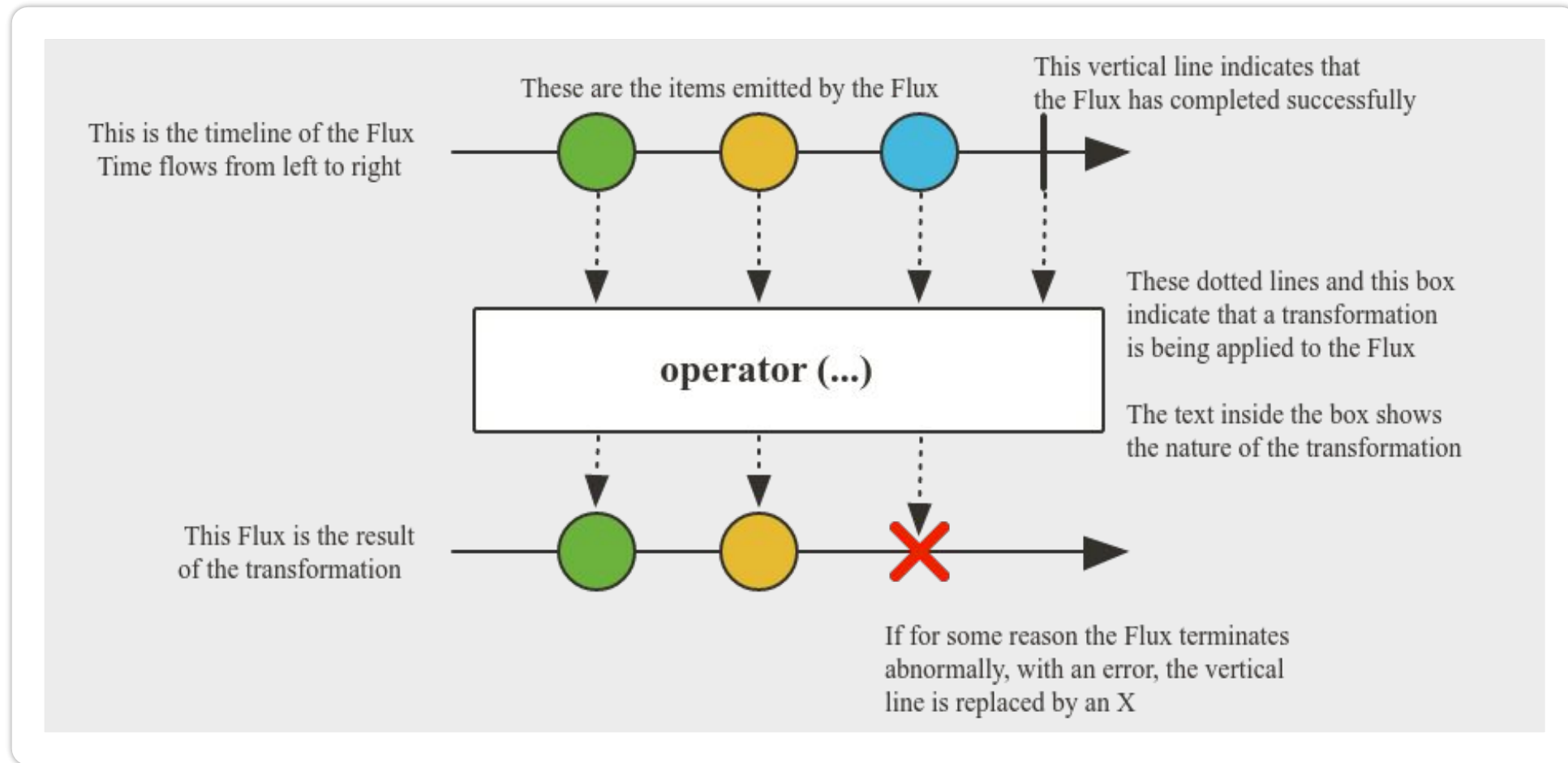
- **L'ingestion de données distribuées à haut débit** à partir d'une variété de sources d'entrée dans un store big data tel que HDFS ou Splunk.
- **Analyse en temps réel** au moment de l'ingestion, par exemple, collecte de métriques et comptage de valeurs.
- **Gestion du flux** via des batchs combinant des interactions avec des systèmes d'entreprise standard (par exemple, SGBDR) ainsi que des opérations Hadoop (par exemple, MapReduce, HDFS, Pig, Hive ou HBase).
- **Exportation de données à haut débit**, par exemple de HDFS vers un SGBDR ou une base de données NoSQL.
- **Composition** : La création de valeur repose la composition de services, comprenant leur agrégation et les échanges de données basés sur un couplage faible.

Reactor - Concepts

- Nous allons retrouver dans Reactor les même concepts que ceux proposés Java 9: Publisher, Processor, Subscriber
 - Avec un cadre de développement plus rigoureux
 - Reactor supporte différents paradigmes d'exécution
 - Push → `subscription.request(Long.MAX_VALUE)`
 - Pull → `subscription.request(1)`
 - Push-pull → adaptation des rythmes de production / consommation
 - Reactor fournit 2 types de publisher
 - **Flux** → 0..N éléments produits
 - **Mono** → 0..1 éléments produits
 - Avec des opérations de conversion possibles entre les deux
-
- ```
graph TD; Publisher[Publisher] -- "Data flow" --> Op1[Operator]; Op1 -- "Data flow" --> Op2[Operator]; Op2 -- "Data flow" --> Subscriber[Subscriber]; Subscriber -.-> Op2; Op2 -.-> Op1; Op1 -.-> Publisher;
```



## Reactor – Concepts



# Reactor – Examples

- **Flux**

- `Flux<String> stream = Flux.just("Hello", "world");`
- `Flux<Integer> stream = Flux.fromArray(new Integer[]{1, 2, 3});`
- `Flux<Integer> stream = Flux.fromIterable(Arrays.asList(9, 8, 7));`

- **Mono**

- `Mono<String> stream = Mono.just("One");`
- `Mono<String> stream = Mono.justOrEmpty(null);`
- `Mono<String> stream = Mono.justOrEmpty(Optional.empty());`

```
Flux.range(1, 10)
 .subscribe(data -> System.out.println(data),
 err -> err.printStackTrace(),
 () -> System.out.println("Completed"));
```

**Exercise:** Reprendre l'exercice de la chaîne de transformation en Flux / Mono

## request(n) : la demande pilote tout

- Le Subscriber demande explicitement n éléments ; le Publisher n'en émet pas davantage.
- Pas de demande = pas de données. Le flux est tiré (pull), pas poussé aveuglément.
- C'est ce protocole qui rend le backpressure natif — on y reviendra en Partie 2.

```
flux.subscribe(
 data -> log.info("next: {}", data), // onNext
 err -> log.error("boom", err), // onError
 () -> log.info("done") // onComplete
);
```

### TROIS PARADIGMES

push → request(MAX\_VALUE)

pull → request(1)

push-pull → on adapte les rythmes

# Rien ne se passe sans souscription

**Assembly-time** on décrit le pipeline (aucune exécution). **Subscription-time** on souscrit → ça s'exécute.

```
Flux<Integer> f = Flux.range(1, 3)
 .doOnNext(i -> log.info("side effect"));
// ici : RIEN n'a été logué – f n'est qu'une recette

f.subscribe(); // maintenant les effets s'exécutent
```

## CONSÉQUENCE

Un Flux est réutilisable et composable comme une valeur.

Les effets de bord ne partent qu'à la souscription — testable.

- Corollaire : c'est le point de souscription qui déclenche, en remontant toute la chaîne.

# Reactor face aux alternatives

## CompletableFuture

Asynchrone, mais une seule valeur, pas de backpressure, composition limitée.

## RxJava

Même famille (Reactive Streams), en déclin côté Spring ; Reactor est la voie Spring.

## Kotlin Coroutines

Alternative ergonomique en Kotlin ; ce cours reste en Java.

## Reactor

Cœur de WebFlux et de Spring Data réactif — c'est notre socle.

# 03

## Créer des Mono/Flux

Des fabriques simples aux sources programmatiques — et l'API Sinks.



# Mono – Création

- **Méthodes**

- Mono.just(element): à utiliser pour des données existantes
- Mono.fromSupplier(lambda): à utiliser pour des données à calculer
- Mono.fromRunnable(runnable): à utiliser pour des données asynchrones
- Mono.fromCallable(callable): à utiliser pour des données à calculer
- Mono.fromFuture(completableFuture): à utiliser pour des données asynchrones
- Mono.empty()
- Mono.error(err)

- **Exercices :**

- Compléter la classe MonoCreate
- Tester les différentes méthodes de création et observer les différences
- Compléter et exécuter le code de MonoSupplier



## Mono – Observations

- Dans le code précédent, on peut observer que les 3 pipelines sont exécutées dans le thread **Main**
  - On observe en conséquence un comportement non bloquant
- Reactor permet d'exécuter les publishers de manière asynchrone selon différentes stratégies
  - Il faut dans ce cas veiller à bloquer le thread main jusqu'à ce que les threads créés se terminent
  - Il est possible également dans ce cas de bloquer le thread main jusqu'à la complétion de threads créés (`Mono::block`)
  - Cela crée implicitement un souscripteur qui se bloque en attente du résultat du publisher
  - À éviter autant que possible car cela va à l'encontre de la philosophie réactive
-

# Flux – Création

- **À partir d'éléments**

- Flux.just(elements...)
- Flux.from(publisher)
- Flux.fromIterable(it)
- Flux.fromStream(stream)
- Flux.empty()
- Flux.error(throwable)

- **Génération**

- Flux.range(start, count)
- Flux.interval(duration)

- **Exercice:**

- Tester les différentes méthodes

## defer & fromCallable : par souscription

```
Mono<User> requestUser(String sessionId) {
 return Mono.defer(() ->
 isValid(sessionId)
 ? Mono.fromCallable(() -> load(sessionId))
 : Mono.error(new InvalidSession()));
}
```

### POURQUOI DEFER ?

Sans defer, la condition serait évaluée à l'assemblage, une fois pour toutes.

Avec defer, elle est ré-évaluée à chaque souscription.

- fromCallable enveloppe un appel synchrone (potentiellement bloquant) en Mono.
- On l'isolera sur boundedElastic en Partie 2 — ici on retient seulement l'approche Lazy.

# Flux infini

- Reactor fournit la possibilité de créer des flux infinis que l'on peut arrêter selon certaines conditions, de différentes manières:
  - `Flux::create(sink)`
  - `Flux::generate(synchronousSink)`, stateless, une seule émission autorisée dans la définition, mais qui est exécutée à l'infini (loop implicite)
  - `Flux::generate(stateSupplier, biFunction)`, idem mais statefull
  - `Flux::push`, pareil au generateur mais un seul thread par émission
- **Exercice:** Compléter la classe `FluxGenerate`

# Les Sinks

```
// pousser des éléments depuis du code impératif
Sinks.Many<MetricSample> sink =
 Sinks.many().multicast().onBackpressureBuffer();

Flux<MetricSample> flux = sink.asFlux();
sink.tryEmitNext(sample); // thread-safe
```

## À NE PLUS ÉCRIRE

Les Processor (DirectProcessor, EmitterProcessor...) sont dépréciés. L'API Sinks les remplace depuis Reactor 3.5.

- Le concept de Sink : En programmation réactive, un Sink (ou "déversoir") est une passerelle qui permet d'alimenter un flux de manière programmatique et thread-safe depuis le monde impératif (par exemple, depuis un listener d'événements, une API REST classique ou un thread séparé).
  - **.many()** : Vous indiquez que ce puits va générer une suite de plusieurs signaux (onNext), et non un signal unique (Mono).
  - **.multicast()** : C'est un choix d'architecture crucial. Le mode multicast signifie que les éléments émis dans le sink seront partagés entre plusieurs abonnés (Subscribers). Si un nouveau composant s'abonne au Flux résultant, il recevra les métriques émises après son abonnement. Les abonnés partagent la même source de données en temps réel.
  - **.onBackpressureBuffer()** : C'est la stratégie de gestion de la contre-pression (backpressure). Si un ou plusieurs abonnés sont trop lents pour traiter les données (MetricSample) par rapport au rythme d'émission, Reactor va stocker temporairement les éléments dans une file d'attente (un buffer en mémoire) de taille illimitée par défaut.

# 04

## Transformer

Les opérateurs du quotidien — et le choix flatMap vs concatMap.



# Opérateurs – Présentation Générale

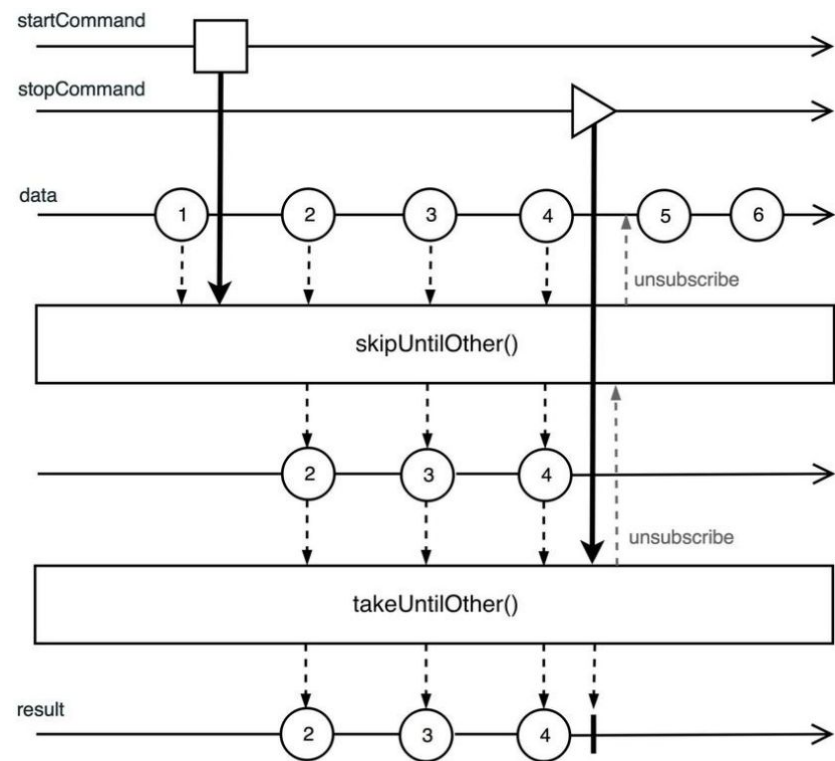
- Reactor nous facilite le traitement des flux par un grand nombre d'opérateurs, regroupés par catégories
  - Map
  - Filtre (take, takeLast, takeUntil, ...)
  - Collection (just, collect, collectSortedList,...)
  - Réduction (any, all, count, ...)
  - Combinaison (concat, merge, zip, ...)
  - Batch (buffer et variants)
  - Cf. <https://projectreactor.io/docs/core/release/api/reactor/core/publisher/Flux.html>

Pour savoir quel opérateur choisir:

<https://projectreactor.io/docs/core/release/reference/#which-operator>

# Opérateurs – Exemple

```
Flux.range(0,100)
 .flatMap(v -> Mono.fromCallable(() -> {
 Thread.sleep(500);
 return v;
 }))
 .skipUntilOther(Mono.delay(Duration.of(5, ChronoUnit.SECONDS)))
 .takeUntilOther(Mono.delay(Duration.of(10, ChronoUnit.SECONDS)))
 .subscribe(System.out::println);
```





# Opérateurs Principaux

- **Take(n)**: retourne un flux contenant n éléments
  - Attention à l'utilisation avec un flux infini
- **Map(function)**: applique une fonction de transformation
- **Filter(predicate)**: filtre selon le prédicat passé en paramètre
- **Handle(biConsumer)**: map + filter
- **LimitRate(num, pourcentage)**: permet de limiter la taille du pipeline entre le publisher et le subscriber
- **DelayElements(duration)**: permet de produire avec une certaine latence
- **Timeout(duration, fallback)**: permet de limiter l'attente sur le publisher et de fournir un autre publisher le cas échéant
- **DefaultIfEmpty**: valeur à retourner si absence de donnée(s)
- **SwitchIfEmpty**: remplace le publisher si absence de donnée(s)
- **Transform**: permet de capitaliser / réutiliser des opérateurs complexes
- **SwitchOnFirst**: permet de remplacer un flux sur la base du premier élément transmit

## map & filter : le pipeline d'ingestion

```
// Pulse – normalisation des échantillons
raw.filter(s -> s.value() >= 0) // capteur valide
 .map(s -> s.withValue(round2(s.value())))
 .map(s -> s.withName(prefix(s.name())));
```

- map : transformation 1→1, synchrone, sans changement de cardinalité.
- filter : ne laisse passer que ce qui satisfait le prédicat.
- Famille de **filtres** : take, takeUntil, skip, distinct, ... ; **réduction** : reduce, count, all/any.

# Hooks

- Reactor permet d'ajouter des hooks aux événements associés à la publication
- Ces méthodes sont généralement de la forme doOnXXXX
  - XXXX représentant l'événement sur lequel on désire ajouter un hook
- **Exercice:** Compléter la classe FluxEvents afin de tester ces différents hooks

# Mettre à plat des publishers : flatMap vs concatMap

```
// concurrent, ordre NON garanti
samples.flatMap(s ->
 directory.regionOf(s.agentId()),
 /*concurrency*/ 8);
```

```
// séquentiel, ordre PRÉSERVÉ
samples.concatMap(s ->
 directory.regionOf(s.agentId())
);
```

## CAS DIFFÉRENTIEL

flatMap entrelace les appels → débit élevé, mais l'ordre de sortie n'est pas celui d'entrée.

concatMap sérialise → ordre garanti, latence cumulée.

Règle : flatMap quand l'ordre n'importe pas (et borner la concurrence) ; concatMap quand il importe.

**Exercice:** Compléter la classe FluxFlatMap et observer le résultat

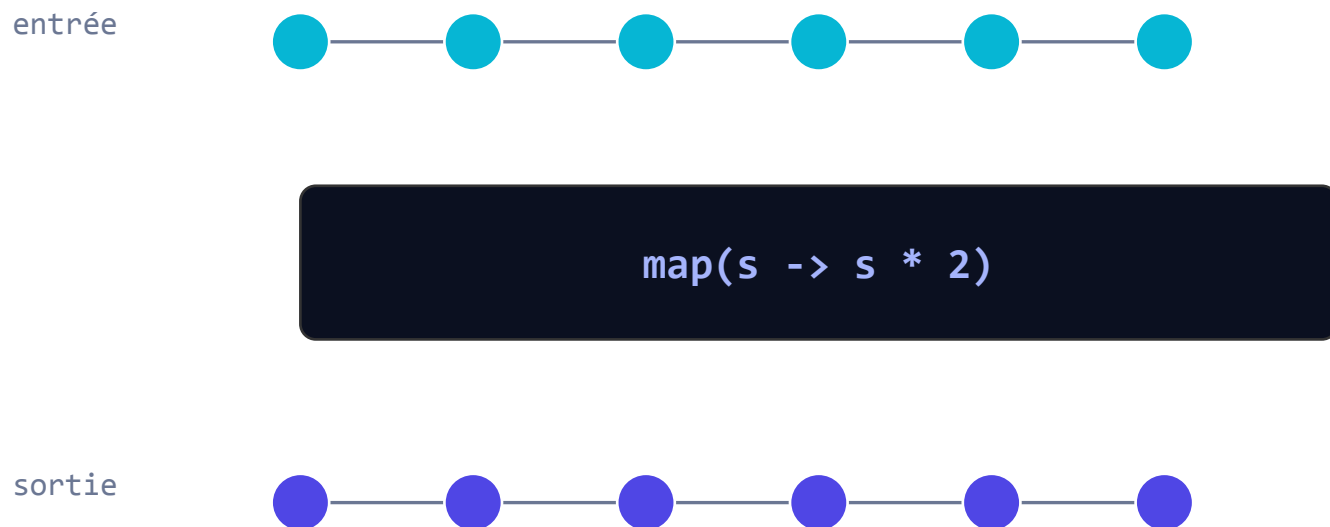
## Réduction et combinaison de flux

```
samples.collectList(); // Flux<T> -> Mono<List<T>>
samples.reduce(0.0, (a, s) -> a + s.value());

Flux.merge(a, b); // entrelacés, dès qu'ils émettent
Flux.concat(a, b); // a en entier, puis b
Flux.zip(a, b, (x, y) -> combine(x, y));
```

- merge : non ordonné, concurrent ; concat : ordonné, séquentiel (même opposition que flatMap/concatMap).
- zip : appariement deux à deux ; combineLatest : recombine à chaque nouvelle émission.
- **Exercice:** Compléter la classe MultiCastFlux

# Le diagramme en marbles



- Le temps va de gauche à droite ; chaque marble est un élément, la barre finale = onComplete.
- La doc Reactor associe un diagramme à chaque opérateur — réflexe : « which operator do I need ? ».

# À vous : bootstrap & premier flux testé

À faire maintenant — fin de matinée (jour 1)

- Squelette Maven + pulse-common : record MetricSample, IngestionPipeline.normalize (filter/map).
- Premier test avec StepVerifier (approfondi juste après, en « Tester les flux »).



# 05

## Gérer les erreurs

L'erreur est un signal — on la traite dans le flux, pas autour.





# L'erreur est un signal terminal

- **onError** est un signal terminal : il interrompt le flux nominal et ferme la séquence.
- On ne l'attrape pas avec try/catch autour d'un block() — on le traite dans le pipeline.
- Les opérateurs d'erreur produisent un flux de repli, conformément à la logique métier.

## LA BOÎTE À OUTILS

**onErrorReturn** — valeur de repli

**onErrorResume** — flux de repli

**onErrorMap** — traduire l'exception

**retryWhen** — re-souscrire

**timeout** — borner l'attente

## Return, resume, map

```
directory.regionOf(id)
 .onErrorReturn(Region.UNKNOWN) // valeur

 .onErrorResume(ex -> cache.lookup(id)) // flux

 .onErrorMap(IOException.class,
 ex -> new DirectoryDownException(ex));
```

- **onErrorReturn** : une valeur constante de secours.
- **onErrorResume** : bascule vers un autre Publisher (cache, défaut calculé...).
- **onErrorMap** : enrichit/traduit l'exception sans changer le flux.

## retryWhen(Retry.backoff) & timeout

```
directory.regionOf(id)
 .timeout(Duration.ofSeconds(2))
 .retryWhen(Retry.backoff(3,
 Duration.ofMillis(200)))
 .onErrorReturn(Region.UNKNOWN);
```

### À NE PLUS ÉCRIRE

retryBackoff(...) a été supprimé.

On passe par **retryWhen(Retry.backoff(...))** (reactor.util.retry.Retry).

- Retry.backoff ajoute un délai croissant + jitter entre tentatives — protège l'amont.
- timeout transforme une attente trop longue en erreur, qu'on peut alors retenter ou replier.

**Exercice:** Compléter la classe FluxErrors afin de tester les différentes stratégies

# 06

## Tester les flux

StepVerifier souscrit pour vous — et le temps virtuel rend les délais gratuits.



## Vérifier le contrat d'un flux

```
StepVerifier.create(
 pipeline.normalize(Flux.just(s1, s2neg, s3)))
 .expectNext(expected1)
 .expectNext(expected3) // s2neg filtré
 .verifyComplete();
```

- StepVerifier souscrit pour vous et vérifie la séquence de signaux, pas une List.
- expectNext / expectError / verifyComplete décrivent le contrat attendu, étape par étape.

**Exercice:** Tester chacun des publishers préalablement définis

# Tester les délais sans attendre

```
StepVerifier.withVirtualTime(() ->
 pipeline.enrich(flux, flakyDirectory))
 .thenAwait(Duration.ofSeconds(1)) // back-off
 .expectNextCount(3)
 .verifyComplete();
```

## POURQUOI UN SUPPLIER ?

withVirtualTime prend un Supplier<Publisher> : l'horloge virtuelle doit exister avant l'assemblage du flux temporel.

- Les 200 ms × 3 de back-off du lab Partie 1-2 sont traversés en quelques millisecondes.

## Démontrer le backpressure en test

```
StepVerifier.create(flux, 0) // demande initiale 0
 .expectSubscription()
 .expectNoEvent(Duration.ofMillis(100)) // rien
 .thenRequest(2)
 .expectNextCount(2) // puis 2
 .thenCancel()
 .verify();
```

- À demande initiale 0, aucune donnée n'arrive : le flux est tiré, pas poussé.
- `thenRequest(n)` libère exactement `n` éléments — le protocole `request(n)` rendu visible.
- Pont direct vers Partie 2 : Schedulers, hot/cold et stratégies `onBackpressure*`.

# À vous : opérateurs, erreurs & temps virtuel

À faire maintenant — fin d'après-midi (jour 1)

- Enrichissement flatMap (concurrency bornée) + retryWhen(Retry.backoff) + timeout.
- Test en temps virtuel (StepVerifier.withVirtualTime), sans attendre l'horloge réelle.





# Partie 1 — l'essentiel

- Le réactif optimise le débit sous charge, pas la latence unitaire — et aujourd'hui, on le pose face à Loom.
- Rien ne s'exécute tant qu'on ne souscrit pas ; request(n) pilote le rythme.
- flatMap (concurrent, borné) vs concatMap (ordonné) : un choix structurant.
- L'erreur est un signal : resume / retryWhen(Retry.backoff) / timeout dans le flux.
- StepVerifier + temps virtuel : on teste contrat et délais sans attendre.



# Spring Reactor

Partie 2 — Concurrency, threads & flux à chaud



# Les six étapes du module

01

**Modèle d'exécution**

`Schedulers • pools`

02

**publishOn / subscribeOn**

`où tourne quoi`

03

**Cold & hot**

`ConnectableFlux • share`

04

**Combiner & cadencer**

`merge • zip • window`

05

**Backpressure**

`buffer • drop • latest`

06

**Debugger un pipeline**

`checkpoint • agent`

**Hier** les flux étaient purs et mono-thread. Aujourd'hui : sur quel thread tourne quoi, et que faire quand le producteur va plus vite que le consommateur.

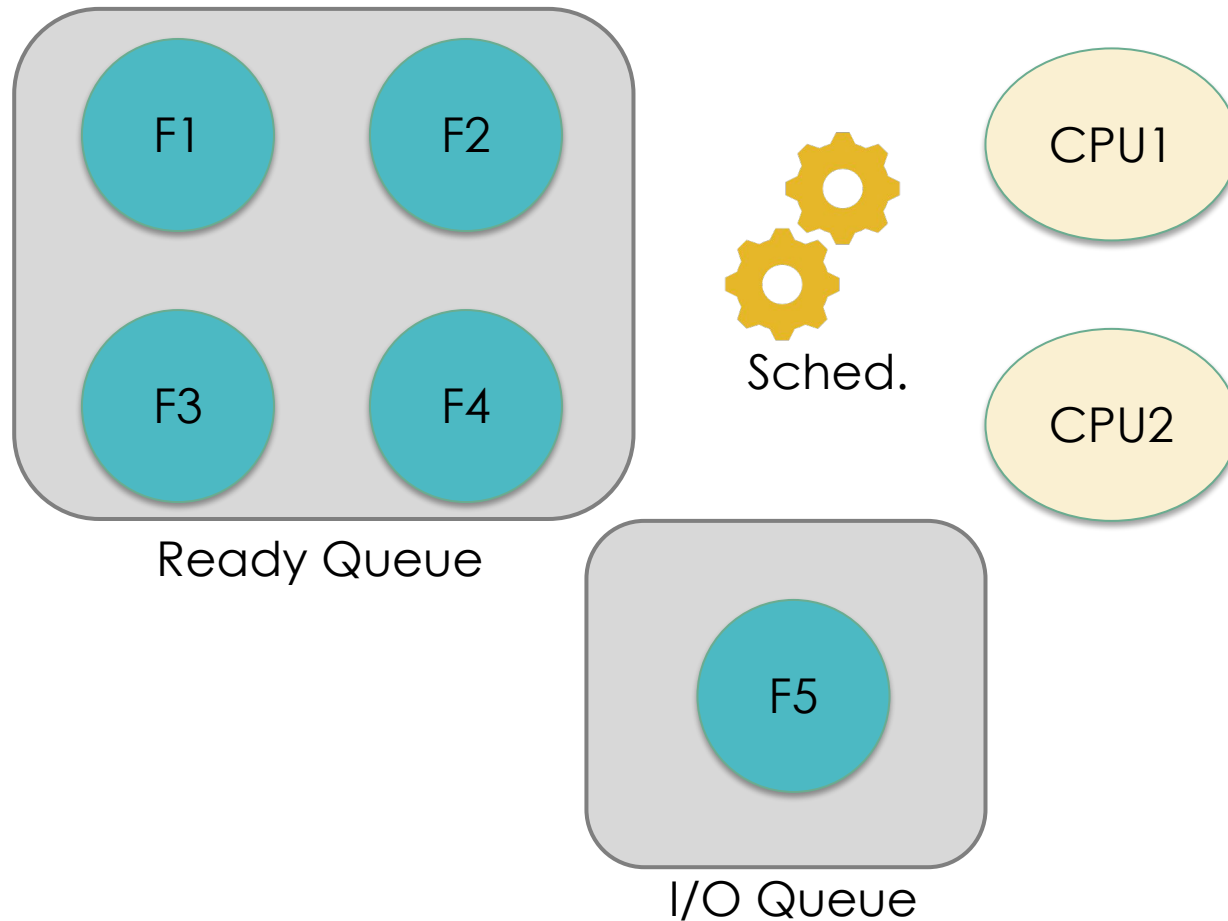
# 01

## Modèle d'exécution & Schedulers

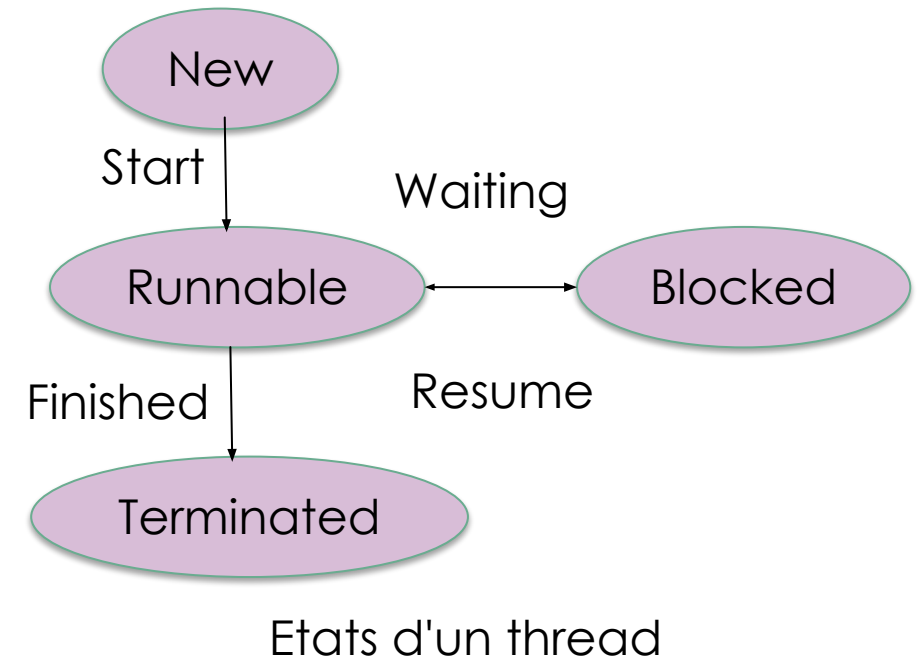
Par défaut, tout tourne sur le thread qui souscrit — jusqu'à ce qu'on en décide autrement.



# Concurrence en Java



- Différentes stratégies d'ordonnancement
  - Pour maximiser les débits
  - Pour maximiser l'égalité d'accès aux ressources
  - Pour minimiser les temps d'attente
  - Pour minimiser la latence



# Concurrence en Java

- Lors de l'exécution d'un programme
  - La JVM exécute un thread principal (main)
  - Le thread principal peut exécuter d'autres threads (enfants)
  - Un thread peut exécuter d'autres threads (arborescence)
  - Un thread doit attendre que ses enfants soient terminés avant de se terminer (join)
- En plus des 4 états vus dans la diapositive précédente, java ajoute deux états qui sont des variants de l'état Blocked
  - Waiting – un thread attend qu'un autre thread ait fini une action
  - Timed\_Waiting – Même chose que waiting mais dans un temps borné
  - La méthode statique `Thread.currentThread()` renvoie une référence vers le thread en cours d'exécution
-

# Sur quel thread tourne un pipeline ?

- **Par défaut**, un pipeline s'exécute sur le thread qui appelle `subscribe()` — pas de magie, pas de thread caché.
- Reactor est concurrency-agnostic : il ne change de thread que si vous le demandez.
- On change de contexte d'exécution avec deux opérateurs seulement : `publishOn` et `subscribeOn`.
- Un Scheduler abstrait un pool de threads ; on le passe à ces opérateurs.
- **Exercice**: Vérifier cette affirmation en affichant le nom du thread sur l'un des Flux que vous avez implementé.

```
Flux.range(1, 3)
 .map(i -> { log(thread()); return i; })
 .subscribe();
// => tout sur le thread appelant (ex. main)
```

## À RETENIR

Aucun opérateur n'introduit d'asynchronisme tout seul.  
Le threading est explicite — c'est une force pour le raisonnement.

# Quel pool pour quel travail?

`boundedElastic()`

Isoler des appels BLOQUANTS (JDBC legacy, I/O fichier). Pool borné.

`parallel()`

Travail CPU-bound non bloquant. Un worker par cœur.

`single()`

Une tâche à la fois, faible latence, ordre garanti.

`immediate()`

Pas d'ordonnancement — exécute sur le thread courant.

`fromExecutorService(...)`

Pont vers un `ExecutorService` existant (y compris virtual threads).

**SUPPRIMÉ**

`Schedulers.elastic()` n'existe plus (pool non borné, fuite de threads) — utilisez `boundedElastic()`.



# CPU vs bloquant — ne pas confondre

```
// CPU-bound : parallel()
flux.parallel()
 .runOn(Schedulers.parallel())
 .map(this::hash) // calcul pur
 .sequential();
```

```
// bloquant : boundedElastic()
flux.flatMap(id ->
 Mono.fromCallable(() -> jdbc.load(id))
 .subscribeOn(Schedulers
 .boundedElastic()), 8);
```

## CAS DIFFÉRENTIEL

Bloquer le pool `parallel()` (un worker par cœur) gèle le calcul de toute l'appli.

Règle : `parallel()` pour du CPU non bloquant ; `boundedElastic()` pour isoler du bloquant — jamais l'inverse.

# Choisir en une phrase

- « Je fais un calcul lourd, sans I/O » → `parallel()`.
- « J'appelle du code bloquant que je ne peux pas rendre réactif » → `boundedElastic()`.
- « J'ai besoin d'un seul thread ordonné » → `single()`.
- « Je veux brancher mon propre pool » → `fromExecutorService(...)`.
- Dans le doute sur du non bloquant : ne rien faire — le thread appelant suffit souvent.

# 02

## publishOn vs subscribeOn

Deux opérateurs, deux portées opposées. La confusion la plus courante de Reactor.



## Agit sur toute la chaîne en amont

- **subscribeOn** fixe le thread où la SOUSCRIPTION démarre — donc où la source émet.
- Sa position dans la chaîne n'a pas d'importance : il affecte tout l'amont, jusqu'à la source.
- Un seul subscribeOn compte vraiment (le plus proche de la source l'emporte).

```
source
 .map(...) // sur boundedElastic
 .filter(...) // sur boundedElastic
 .subscribeOn(
 Schedulers.boundedElastic());
```

### IMAGE MENTALE

subscribeOn = « où je branche la prise ». Toute la guirlande en amont s'allume sur ce pool.

## Bascule l'aval à partir de son point

- **publishOn** change le thread de tout ce qui se trouve EN AVAL
- Sa position est décisive : on peut enchaîner plusieurs publishOn pour des bascules successives.
- Usage typique : faire le travail bloquant/lent sur un pool, livrer le résultat sur un autre.

```
source // thread appelant
 .map(this::decode)
 .publishOn(Schedulers.parallel())
 .map(this::transform) // sur parallel
 .subscribe();
```

### LA RÈGLE QUI COLLE

subscribeOn → AMONT, indépendant de la position.  
publishOn → AVAL, dépendant de la position.

## Combiner les deux — et leur coût

```
Flux.defer(() -> blockingSource()) // I/O
 .subscribeOn(Schedulers.boundedElastic()) // lecture isolée
 .publishOn(Schedulers.parallel()) // traitement
 .map(this::heavyTransform)
 .subscribe(this::emit);
```

- Chaque changement de thread a un coût (handoff + cache). N'en mettez pas « au cas où ».
- Mesurez : souvent un seul subscribeOn bien placé suffit, sans publishOn.
- **Exercice**: Tester l'influence des deux méthodes sur un Flux de votre choix en affichant le nom du thread exécutant.

# 03

## Cold & hot publishers

Le flux rejoue-t-il tout pour chaque abonné, ou partage-t-il une seule émission ?



## Chaque abonné repart de zéro

- Un cold publisher ne produit rien tant qu'il n'a pas d'abonné, et rejoue toute la séquence pour chacun.
- C'est le cas par défaut de la plupart des fabriques (just, range, defer, fromIterable).
- Deux souscriptions = deux exécutions indépendantes.

```
Flux<Integer> cold =
 Flux.defer(() -> Flux.range(0, 5));

cold.subscribe(...); // 0..4
cold.subscribe(...); // 0..4 (re-joué)
```

### INTUITION

Cold = un disque : chacun le relance depuis le début, à son rythme.



## Une source partagée par tous

- Un hot publisher émet indépendamment des abonnés ; les retardataires manquent le passé.
- ConnectableFlux transforme un cold en hot : publish() (ou replay()) puis connect().
- Cas Pulse : un seul flux d'ingestion alimente plusieurs consommateurs (dashboard, alertes).
- **Exercice:** Reprendre un Flux de votre choix et le transformer en hot publisher avec la méthode share() et tester

```
ConnectableFlux<Integer> hot =
 Flux.range(0, 5).publish();

hot.subscribe(a);
hot.subscribe(b);
hot.connect(); // démarre l'émission
```

# autoConnect, refCount, share, cache

`connect()`

Démarrage manuel, une fois prêts tous les abonnés.

`autoConnect(n)`

Démarre après le n-ième abonné, ne s'arrête jamais.

`refCount(n)`

Démarre au n-ième abonné, s'arrête quand ils partent tous.

`share()`

Raccourci : `publish().refCount(1)` — partage à la volée.

`cache()` / `cache(d)`

Rejoue aux nouveaux abonnés ce qui a déjà été émis.

**Exercice:** Modifier la classe `FluxDelay` afin d'illustrer le concept de Hot Publisher

# 04

## Combiner & cadencer les flux

Assembler plusieurs sources, et jouer avec le temps.



## merge & concat

```
// entrelacés, dès qu'ils émettent
Flux.merge(agentA, agentB);

// concurrent, ordre non garanti
```

```
// a en entier, PUIS b
Flux.concat(history, live);

// séquentiel, ordre préservé
```

- Même opposition que flatMap/concatMap : merge = débit, concat = ordre.
- mergeSequential : souscrit en parallèle mais ré-ordonne la sortie.
- **Exercice**: Compléter la classe MultiCastFlux afin de tester ces méthodes

## zip & combineLatest

```
// appariement 1-1, attend les deux
Flux.zip(prices, volumes,
 (p, v) -> p.times(v));

// recombine à CHAQUE nouvelle émission
Flux.combineLatest(cpu, mem,
 (c, m) -> new Load(c, m));
```

- **zip** : cadence sur le plus lent ; un élément non apparié attend son binôme.
- **combineLatest** : émet dès qu'une source bouge, avec la dernière valeur des autres.

- **Exercice**: Compléter la classe MultiCastFlux afin de tester ces méthodes

## interval, delayElements, elapsed

```
Flux.interval(Duration.ofMillis(100)) // tic régulier
 .take(5);

samples.delayElements(Duration.ofMillis(50));

samples.elapsed() // Tuple2<millis, valeur>
```

- interval est un flux chaud cadencé par le temps — testable en temps virtuel (Partie 1).
- elapsed / timestamp instrumentent le flux pour mesurer les cadences.
- **Exercice:** Tester ces méthodes sur un Flux de votre choix

## buffer & window

```
// paquets de List<T>
samples.buffer(
 Duration.ofSeconds(1));
// -> Flux<List<MetricSample>>
```

```
// sous-flux Flux<Flux<T>>
samples.window(
 Duration.ofSeconds(1));
// agrégation à la volée par fenêtre
```

- buffer accumule puis émet une liste ; pratique pour des écritures par lots (DB, Kafka).
- window ouvre des sous-flux qu'on traite en continu — agrégations glissantes côté Pulse.
- **groupBy** diffère de la méthode window dans la mesure où elle renvoie différents flux créés selon certains critères
- **Exercice**: Tester ces méthodes sur un Flux de votre choix

# 05

## Backpressure en profondeur

Producteur rapide, consommateur lent : qui ralentit, et que perd-on ?





## Producteur rapide, consommateur lent

- **Rappel** : le consommateur demande (request(n)) ; un producteur qui respecte la demande ne déborde jamais.
- **Problème** : certaines sources ne peuvent pas ralentir (interval, capteurs, événements externes).
- Il faut alors décider explicitement quoi faire du trop-plein — c'est la stratégie de backpressure.

```
Flux.interval(Duration.ofMillis(1)) // rapide
 .onBackpressureBuffer()
 .publishOn(Schedulers.boundedElastic())
 .concatMap(this::slowSink, 1) // lent
 .subscribe();
```

# Que faire du trop-plein?

**onBackpressureBuffer**

File d'attente. Zéro perte, mais risque mémoire si non bornée.

**onBackpressureDrop**

Jette les éléments qui ne peuvent être traités tout de suite.

**onBackpressureLatest**

Ne garde que le dernier ; les plus anciens sont écartés.

**onBackpressureError**

Émet une erreur d'overflow — échoue vite, fail-fast.

## CAS DIFFÉRENTIEL

Buffer = aucune perte, mémoire qui gonfle. Latest/Drop = mémoire bornée, perte assumée.

Le choix dépend du métier : une métrique périmée se jette (latest) ; un ordre de paiement, jamais (buffer + alerte).

## limitRate & prefetch

```
// borne la demande en amont par paquets de 64
flux.limitRate(64);

// la plupart des opérateurs ont un prefetch (256)
flux.flatMap(this::call, /*concurrency*/ 8,
 /*prefetch*/ 32);
```

- limitRate transforme une demande illimitée (request(MAX)) en demandes bornées et renouvelées.
- Le prefetch contrôle combien d'éléments un opérateur demande d'avance — compromis débit/mémoire.

## boundedElastic ou virtual threads ?

- boundedElastic existe pour isoler du bloquant sur un pool borné, sans figer parallel().
- Avec Java 21+, on peut brancher un pool de virtual threads via fromExecutorService.
- Mais Reactor n'exécute pas ses opérateurs sur des virtual threads par défaut — la nuance compte.

```
Schedulers.fromExecutorService(
 Executors
 .newVirtualThreadPerTaskExecutor());
```

### ON MESURERA EN Partie 4

Le vrai arbitrage réactif vs MVC+Loom se tranche aux chiffres, sur le jumeau Pulse — pas ici, pas au dogme.

# 06

## Debugger un pipeline

La frontière asynchrone perd la stacktrace : Reactor donne des outils pour la retrouver.



## checkpoint() & log()

```
ingestion
 .map(this::normalize)
 .checkpoint("après normalisation") // léger
 .flatMap(this::enrich)
 .log("pulse.ingestion") // tous signaux
 .subscribe();
```

- `checkpoint(label)` marque un point d'assemblage ; en cas d'erreur, la trace pointe ce label.
- `log()` journalise `onSubscribe` / `request` / `onNext` / `onComplete` — verbeux, pour l'investigation.

- **Exercice:** Tester la méthode `log` sur un Flux de votre choix

# ReactorDebugAgent vs onOperatorDebug

## REACTORDEBUGAGENT – PROD OK

Instrumentation bytecode au démarrage.

Coût négligeable.

Active partout, en continu.

```
ReactorDebugAgent.init();
```

## HOOKS.ONOPERATORDEBUG() – DEV ONLY

Capture la stacktrace à CHAQUE assemblage.

Coût élevé.

À réserver au développement.

```
Hooks.onOperatorDebug();
```

- Les deux reconstituent la chaîne d'opérateurs ; en prod, on garde l'agent, jamais le hook global.

# La stacktrace assemblée

- Sans aide, une erreur asynchrone montre la pile du thread d'exécution, pas le code qui a assemblé le flux.
- checkpoint et l'agent ajoutent une section « Assembly trace » qui pointe vos lignes.
- Réflexe : lire d'abord la section assemblée, pas la pile interne de Reactor.

```
Error has been observed at the
following site(s):
 *__checkpoint(après normalisation)
Original Stack Trace:
 at pulse.IngestionPipeline...
```

**Exercice de synthèse:** Compléter la classe Titanic



# À vous : schedulers & backpressure

À faire à l'issue de ce module

- `publishOn` / `subscribeOn` prouvés par assertion ; offload du bloquant sur `boundedElastic`.
- Producteur rapide / consommateur lent : démontrer les stratégies `onBackpressure*`.



## À RETENIR

# Concurrence & backpressure — l'essentiel

- Par défaut tout tourne sur le thread qui souscrit ; on change avec `publishOn` (aval) et `subscribeOn` (amont).
- `parallel()` pour le CPU, `boundedElastic()` pour isoler du bloquant — jamais l'inverse.
- Cold = rejoué par abonné ; hot = partagé (`ConnectableFlux`, `share`, `cache`).
- Backpressure : `buffer` (zéro perte, mémoire) vs `latest/drop` (borné, perte assumée) — choix métier.
- Debug : `checkpoint()` + `ReactorDebugAgent` en prod ; `Hooks.onOperatorDebug()` en dev seulement.



Cap sur **WebFlux** — exposer ce pipeline en API réactive, et le comparer au jumeau MVC + Loom.

JANUS ACADEMY • FORMATION

# Spring Reactor

WebFlux & WebClient



Spring Boot 4 • Netty • SSE/WebSocket • différentiel MVC+Loom

# Les six étapes du module

01

WebFlux vs MVC

Netty • event-loop • Loom

02

Construire l'API

annoté & fonctionnel

03

WebClient

appeler d'autres services

04

Sérialisation & validation

Jackson 3 • jakarta

05

Temps réel

SSE & WebSocket

06

Le différentiel mesuré

WebFlux vs MVC+Loom

**Fil rouge** — Pulse devient une vraie API : endpoints réactifs, fan-out WebClient, flux temps réel — et son jumeau MVC+Loom pour comparer.

# 01

## WebFlux vs MVC — et Loom

Deux piles d'exécution, trois choix d'architecture.



# Thread-per-request vs event-loop

## SPRING MVC – SERVLET

Pile servlet (Tomcat par défaut).

Un thread dédié par requête.

Bloque pendant l'I/O – thread immobilisé.

Modèle impératif, familier, débogable.

Scalabilité limitée par la taille du pool.

## WEBFLUX – RÉACTIF

Pile Netty (event-loop) par défaut.

Peu de threads, beaucoup de connexions.

Ne bloque jamais : libère le thread sur l'I/O.

Contrôleurs renvoyant Mono / Flux.

Non bloquant de bout en bout – obligatoire.

# Non bloquant de bout en bout

- WebFlux ne tient ses promesses que si TOUTE la chaîne est non bloquante : contrôleur, accès données, appels sortants.
- Un seul appel bloquant sur un thread d'event-loop bloque ce thread pour toutes les requêtes qu'il sert.
- Donc : R2DBC (pas JDBC bloquant), WebClient (pas RestTemplate), et le bloquant inévitable isolé sur boundedElastic.

## LE PIÈGE N°1

Un appel JDBC bloquant directement dans un contrôleur WebFlux gèle l'event-loop.

Symptôme : débit qui s'effondre sous charge, quelques threads Netty bloqués.

# Trois architectures, pas deux

## MVC CLASSIQUE

Thread-per-request bloquant.  
Simple, mais plafonne sous forte charge I/O.

## MVC + VIRTUAL THREADS

Code impératif, threads quasi gratuits.  
Suffit pour la plupart du CRUD I/O-bound.

## WEBFLUX

Non bloquant de bout en bout.  
Gagne sur streaming / backpressure / temps réel.

## CAS DIFFÉRENTIEL – FIL ROUGE

On implémente les mêmes endpoints Pulse en WebFlux ET en MVC+Loom, puis on mesure (section 6).



# Ce que renvoie un contrôleur réactif

```
@GetMapping("/api/alerts")
public Flux<Alert> alerts() { // 0..N
 return alertService.findAll();
}

@GetMapping("/api/alerts/{id}")
public Mono<Alert> byId(@PathVariable String id)
```

## NE JAMAIS

block(), subscribe() ou .toFuture().get() dans un contrôleur.

On retourne le Publisher : le framework souscrit pour vous.

# 02

## Construire l'API réactive

Style annoté ou fonctionnel — deux façons d'exposer les routes.



# @RestController

```
@RestController
@RequestMapping("/api/alerts")
class AlertController {
 @GetMapping
 Flux<Alert> list() { return service.all(); }
 @PostMapping
 Mono<Alert> create(@Valid @RequestBody
 Mono<AlertRule> rule) { ... }
}
```

- Mêmes annotations que MVC : la transition est douce.
- Les types de retour Mono/Flux signalent le pipeline réactif.
- @RequestBody peut être un Mono<T> : corps désérialisé en flux.

# RouterFunction & HandlerFunction

```
@Bean
RouterFunction<ServerResponse> routes(AlertHandler h) {
 return route()
 .GET("/api/alerts", h::list)
 .POST("/api/alerts", h::create)
 .build();
}
```

- Le routage est une valeur : composable, testable, sans annotations.
- Un HandlerFunction prend un ServerRequest, renvoie un Mono<ServerResponse>.

# Le HandlerFunction

```
Mono<ServerResponse> list(ServerRequest req) {
 return ServerResponse.ok()
 .contentType(APPLICATION_JSON)
 .body(service.all(), Alert.class);
}
```

- Tout reste réactif : on assemble la réponse comme un pipeline.
- Le corps est branché sur un Publisher (service.all()), jamais matérialisé.

# Annoté ou fonctionnel ?

## ANNOTÉ

Familiarité MVC, montée en charge d'équipe.  
Conventions Spring, moins de code d'infra.  
Le défaut raisonnable pour la plupart des API.

## FONCTIONNEL

Routage explicite et composable.  
Idéal pour des routes dynamiques / modulaires.  
Tests légers sans contexte web complet.

# 03

## WebClient

Le client HTTP non bloquant — appeler les autres services sans figer un thread.



# Appeler un service

```
WebClient client = WebClient.builder()
 .baseUrl("http://upstream:8080").build();

Flux<Health> health = client.get()
 .uri("/health")
 .retrieve()
 .bodyToFlux(Health.class);
```

- WebClient remplace RestTemplate : 100 % non bloquant.
- retrieve() pour le cas courant ; exchangeToFlux() pour piloter la réponse finement.
- Le côté MVC+Loom utilisera RestClient (bloquant)
  - même API fluide, sémantique différente.



## timeout, retry, backoff

```
client.get().uri("/region/{id}", id)
 .retrieve()
 .bodyToMono(Region.class)
 .timeout(Duration.ofSeconds(2))
 .retryWhen(Retry.backoff(3,
 Duration.ofMillis(200)))
 .onErrorReturn(Region.UNKNOWN);
```

- Les mêmes opérateurs qu'en Partie 1 : timeout, retryWhen(Retry.backoff), onErrorResume.
- On protège l'amont (jitter, plafond de tentatives) et on borne l'attente.

# Clients déclaratifs & résilience native

```
// client HTTP déclaratif (interface)
interface DirectoryClient {
 @GetExchange("/region/{id}")
 Mono<Region> region(@PathVariable String id);
}
```

- @HttpExchange transforme une interface en client HTTP — typé, testable, sans boilerplate.
- Natif vs Resilience4j : natif pour le simple, R4J pour le pointu.

## SPRING FRAMEWORK 7

Circuit breaker intégré pour les Interface Clients ; @Retryable s'applique aux retours réactifs.

Resilience4j pour bulkhead / rate-limiter avancés.

# Clients déclaratifs & résilience native – Utilisation

```
@Configuration
public class ClientConfig {

 @Bean
 public DirectoryClient directoryClient(WebClient.Builder builder) {
 // 1. On configure le WebClient sous-jacent avec l'URL de base
 WebClient webClient = builder.baseUrl("https://api.directory.local").build();

 // 2. On crée un adaptateur pour les HTTP Interfaces
 WebClientAdapter adapter = WebClientAdapter.create(webClient);
 HttpServiceProxyFactory factory = HttpServiceProxyFactory.builderFor(adapter).build();

 // 3. Spring génère l'implémentation de notre interface
 return factory.createClient(DirectoryClient.class);
 }
}
```

## Fan-out / fan-in

```
// interroger N upstreams en parallèle, recombinaison
Flux.fromIterable(upstreams)
 .flatMap(u -> client.get().uri(u.health())
 .retrieve().bodyToMono(Health.class)
 .timeout(Duration.ofMillis(800))
 .onErrorReturn(Health.DOWN), 16)
 .collectList();
```

- flatMap concurrent + timeout par appel : un upstream lent ne bloque pas l'agrégat.
- C'est l'agrégation de santé de Pulse — le cas où le réactif brille.

# 04

## Sérialisation & validation

Jackson 3 par défaut, JSON en flux, et la validation jakarta.



# Sérialiser un flux

```
@GetMapping(value = "/api/metrics", produces = "application/x-ndjson")
Flux<MetricSample> metrics() {
 return service.stream();
}
```

- Spring Boot 4 passe à Jackson 3 par défaut (nouveaux packages, comportement aligné).
- Les records Java se sérialisent nativement — DTOs immuables dans pulse-common.
- Un Flux<T> peut se streamer en JSON ligne par ligne (application/x-ndjson) sans tout matérialiser.

# Contraintes jakarta

```
record AlertRule(
 @NotBlank String metric,
 @Positive double threshold) {}

@PostMapping
Mono<Alert> create(
 @Valid @RequestBody AlertRule rule) { ... }
```

- jakarta.validation (plus javax) ; @Valid déclenche la validation avant le handler.
- Un échec produit un signal d'erreur — traité de façon réactive, pas par exception synchrone.

# Gérer les erreurs proprement

```
@ExceptionHandler({AlertNotFound.class})
ResponseBody<ProblemDetail> handle(AlertNotFound ex) {
 return ResponseEntity.status(NOT_FOUND)
 .body(ProblemDetail.forStatus(NOT_FOUND));
}
```

- ProblemDetail (RFC 7807) standardise le corps d'erreur — natif Spring.
- Les erreurs du pipeline (onError) deviennent des réponses HTTP via les handlers.



# 05

## Temps réel : SSE & WebSocket

Pousser les métriques Pulse vers le navigateur, en continu.



# Server-Sent Events

```
@GetMapping(value = "/api/metrics/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
Flux<ServerSentEvent<MetricSample>> stream() {
 return service.live()
 .map(s -> ServerSentEvent.builder(s)
 .event("metric").build());
}
```

- SSE = flux unidirectionnel serveur → client, sur HTTP simple.
- Un Flux infini se mappe directement en flux d'événements — le cas idéal du réactif.
- Le navigateur consomme via EventSource ; reconnexion automatique.

# Bidirectionnel

```
class MetricsSocket implements WebSocketHandler {
 public Mono<Void> handle(WebSocketSession s) {
 return s.send(service.live()
 .map(m -> s.textMessage(toJson(m))));
 }
}
```

- WebSocket = canal bidirectionnel persistant ;  
send et receive sont des Flux.
- À réserver aux échanges deux-sens  
(commandes client → serveur).

# SSE ou WebSocket ?

## SSE – DÉFAUT

Serveur → client, unidirectionnel.  
HTTP standard, proxies/firewalls amicaux.  
Reconnexion native côté navigateur.  
Parfait pour un flux de métriques.

## WEBSOCKET – SI BESOIN

Bidirectionnel, faible latence.  
Protocole dédié (upgrade HTTP).  
Nécessaire si le client doit aussi pousser.  
Plus lourd à opérer.

# 06

## Le différentiel mesuré

Même contrat, deux implémentations, des chiffres — pas du dogme.



# Deux implémentations, un contrat

- pulse-reactive (WebFlux + R2DBC + WebClient) et pulse-mvc-loom (MVC + virtual threads + JDBC + RestClient).
- Strictement les mêmes endpoints et DTOs (pulse-common) : la comparaison est honnête.
- Gatling tape les deux sous la même charge ; on compare, on ne suppose pas.

## LAB Partie 2-2

C'est exactement ce que vous construisez cet après-midi :

- l'API réactive,
- son jumeau MVC+Loom,
- la simulation Gatling qui les départage.

# Les bonnes métriques

- Débit (req/s) sous charge croissante — pas la latence d'une requête isolée.
- Latence p99 / p999 quand la concurrence monte : c'est là que les modèles divergent.
- Threads et mémoire à charge égale : combien de threads pour tenir N connexions ?
- Comportement à saturation : dégradation douce (réactif/backpressure) vs effondrement.

# Le verdict, honnêtement

## SUR LE CRUD I/O-BOUND

MVC + Loom  $\approx$  WebFlux en débit.

Code impératif plus simple à maintenir.

Souvent, Loom suffit.

## SUR LE STREAMING / FORTE CHARGE

WebFlux prend l'avantage : backpressure, flux continu, empreinte threads minimale.

C'est le terrain où le réactif se justifie.

*Le verdict chiffré et le runbook de décision arrivent en Partie 4.*



# À vous : API WebFlux, jumeau MVC+Loom & Gatling

À faire à l'issue de ce module (toute la PM)

- API réactive + fan-out WebClient + SSE ; jumeau MVC+Loom à contrat identique.
- Premier différentiel mesuré avec Gatling.



Modules pulse-reactive / -mvc-loom / -loadtest • ~3 h • solution : tag lab-j2-2

À RETENIR

# WebFlux — l'essentiel

- WebFlux = Netty + non bloquant de bout en bout ; un seul maillon bloquant ruine le modèle.
- Contrôleurs annotés ou fonctionnels : on retourne le Publisher, on ne block() jamais.
- WebClient (et @HttpExchange) pour les appels sortants ; timeout + retryWhen(Retry.backoff).
- SSE pour pousser un flux serveur → client ; WebSocket si bidirectionnel nécessaire.
- Le différentiel WebFlux vs MVC+Loom se tranche aux mesures — Gatling, cet après-midi.



Cap sur Partie 3 — données réactives (R2DBC), propagation de contexte, puis patterns temps réel.

# Spring Reactor

Données réactives & Context



# Les six étapes du module

**01**

**R2DBC**

le SQL réactif

**02**

**Spring Data R2DBC**

repositories réactifs

**03**

**Transactions & limites**

tx • pagination • BP

**04**

**Le Reactor Context**

pourquoi ThreadLocal casse

**05**

**Propager le contexte**

Micrometer • MDC • sécurité

**06**

**Cache & Mongo**

cache réactif • tailable

**Fil rouge** — Pulse passe du stockage en mémoire à PostgreSQL en R2DBC, et propage un identifiant de trace de l'entrée HTTP jusqu'au log de la requête SQL.

# 01

## R2DBC — le SQL réactif

Un SPI réactif pour les bases relationnelles. Ni JDBC, ni un ORM.



# Reactive Relational DB Connectivity

- R2DBC est un SPI non bloquant pour SQL — l'équivalent réactif de JDBC, pas une couche ORM.
- Drivers dédiés : r2dbc-postgresql, r2dbc-mssql, etc. (le driver JDBC ne convient pas).
- Sans driver réactif, JDBC bloquant gèlerait l'event-loop : R2DBC est la pièce qui manquait au bout-en-bout.

```
ConnectionFactory cf = ConnectionFactories
 .get("r2dbc:postgresql://localhost/pulse");
// SPI bas niveau – on ne l'écrit qu'une fois
```

## EN PRATIQUE

On code rarement le SPI directement.

On passe par DatabaseClient (fluide) ou Spring Data R2DBC (repositories).

# Une requête, une fois, pour comprendre

```
Mono.from(connectionFactory.create())
 .flatMapMany(c -> c.createStatement(
 "SELECT name FROM agent WHERE region=$1")
 .bind("$1", region).execute())
 .flatMap(r -> r.map((row, meta) ->
 row.get("name", String.class)));
```

- Tout est Mono/Flux : connexion, statement, résultat — non bloquant à chaque étage.
- On le montre une fois pour démystifier ; ensuite on monte d'un cran avec Spring Data.

# R2DBC ou JDBC + virtual threads ?

## R2DBC

Non bloquant de bout en bout (obligatoire en WebFlux).  
Backpressure jusqu'à la base.  
Mais : pas d'ORM, pas de relations, plus de code.

## JDBC + LOOM

Garde JPA/Hibernate, relations, cache L1.  
Code impératif familier, virtual threads pour la scalabilité.  
Souvent le bon choix hors streaming.

*Cas différentiel : si le domaine est riche (relations, ORM), JDBC+Loom reste pertinent. R2DBC brille sur le flux et le bout-en-bout.*



# Brancher Pulse sur PostgreSQL

```
application.yml
spring:
 r2dbc:
 url: r2dbc:postgresql://localhost:5432/pulse
 username: pulse
 password: pulse
```

- Starter spring-boot-starter-data-r2dbc + driver r2dbc-postgresql.
- Schéma initialisé par schema.sql (pas de DDL auto type Hibernate).
- Testcontainers fournit une vraie base éphémère en test (lab j4-1).

# 02

## Spring Data R2DBC

Des repositories réactifs — sans le poids d'un ORM.



## Des méthodes qui renvoient Mono/Flux

```
interface AlertRepository
 extends R2dbcRepository<Alert, Long> {

 Flux<Alert> findByMetric(String metric);
 Mono<Alert> findByName(String name);
}
```

- R2dbcRepository fournit save/findById/findAll... en versions réactives.
- Les requêtes dérivées (findByX) sont générées comme en Spring Data classique.
- On retourne directement Mono/Flux au contrôleur — pas de matérialisation.

## Une entité = une table, à plat

```
@Table("alert")
record Alert(
 @Id Long id,
 String metric,
 double threshold) {}
```

### PAS DE MAGIE ORM

Pas de lazy loading, pas de relations gérées, pas de cache de session.

Les jointures se font à la main (DatabaseClient) ou par requête dédiée.

- Mapping plat : un record ↔ une ligne. Les records de pulse-common conviennent.
- C'est proche d'un JdbcTemplate réactif, pas d'Hibernate — on assume la simplicité.

# Le requêtage fluide

```
client.sql("""
 SELECT region, count(*) AS n FROM agent GROUP BY region
""")
 .map((row, m) -> new RegionCount(
 row.get("region", String.class),
 row.get("n", Long.class)))
 .all(); // Flux<RegionCount>
```

- DatabaseClient : pour les requêtes que les repositories ne couvrent pas (agrégats, jointures).
- Toujours réactif : `.all()` → Flux, `.one()/.first()` → Mono.

# Pages et flux

- Pas de `Page<T>` bloquant : on combine une requête de contenu (Flux) et un count (Mono).
- Pour de gros volumes, préférez le streaming (Flux) au paging — le backpressure régule.
- `limitRate` / `prefetch` (vus en Partie 2) contrôlent combien de lignes on tire d'avance.

```
repo.findAllBy(PageRequest.of(page, size))
 .collectList()
 .zipWith(repo.count());
```

# 03

## Transactions, pagination & backpressure

Des transactions réactives — et des limites à connaître.



## @Transactional, en réactif

```
@Transactional
public Mono<Alert> createWithAudit(AlertRule r) {
 return alertRepo.save(toAlert(r))
 .flatMap(a -> auditRepo.save(audit(a))
 .thenReturn(a));
}
```

- Un ReactiveTransactionManager gère la transaction le long du pipeline.
- @Transactional fonctionne sur un retour Mono/Flux ; ou TransactionalOperator pour un contrôle fin.
- La transaction est portée par le Context réactif — d'où la section suivante.



## Ce que R2DBC ne fait pas

### Pas de dirty checking

On sauvegarde explicitement ; aucune synchro auto à la fin de tx.

### Pas de lazy loading

Pas de relations chargées à la demande — requêtes explicites.

### Pas de cache de session

Aucun cache L1 ; chaque accès touche la base.

### Écosystème plus jeune

Moins d'outillage qu'autour de JPA/Hibernate.

# Backpressure de bout en bout

- Un Flux issu de R2DBC respecte la demande : le client SSE lent ralentit la lecture en base.
- C'est l'argument fort du réactif sur les données : la pression remonte du navigateur jusqu'au curseur SQL.
- Avec JDBC+Loom, ce maillon-là n'existe pas — on lit tout, puis on pousse.

## LE FIL ROUGE

Pulse : `/api/metrics/stream` lit la base en flux et pousse en SSE — un seul tuyau régulé du curseur au client.

# 04

## Le Reactor Context

Pourquoi ThreadLocal casse en réactif — et par quoi on le remplace.



# ThreadLocal ne survit pas aux frontières

- MDC (logs), SecurityContextHolder, tracing : tout repose sur des ThreadLocal en monde bloquant.
- En réactif, un pipeline change de thread (publishOn, boundedElastic) : le ThreadLocal du souscripteur n'est plus là.
- Résultat : un traceId posé à l'entrée HTTP disparaît au moment de logger la requête SQL.

## SYMPTÔME TYPIQUE

Logs sans traceId, SecurityContext vide au fond du pipeline, spans de tracing cassés.

La cause n'est pas un bug : c'est le modèle d'exécution multi-thread.

## Un Context porté par la souscription

```
Mono.deferContextual(ctx ->
 log(ctx.get("traceId")) // lecture
)
 .contextWrite(c ->
 c.put("traceId", id)); // écriture
```

- Reactor attache un Context immuable à la souscription, indépendant du thread.
- Particularité : le Context s'écrit en aval et se lit en amont — il remonte la chaîne.
- On écrit avec contextWrite, on lit avec deferContextual / Mono.deferContextual.

## Le Context remonte la chaîne

- contextWrite placé en bas de la chaîne est visible par les opérateurs au-DESSUS de lui.
- Intuition : c'est l'inverse des données (qui descendent). Le Context, lui, remonte vers la source.
- Donc on écrit le Context au plus près du point de souscription (ex. un WebFilter), pour qu'il couvre tout.

```
// WebFilter : pose le contexte pour toute la requête
return chain.filter(exchange)
 .contextWrite(c -> c.put("traceId", traceId));
```

# Immuable, typé, parcimonieux

- Le Context est immuable : chaque put renvoie un nouveau Context, il n'y a pas d'effet de bord.
- Utilisez des clés typées et stables (constantes) plutôt que des chaînes magiques éparpillées.
- N'y mettez que du transverse (trace, sécurité, tenant) — pas vos données métier.
- Plomber manuellement chaque ThreadLocal serait pénible : d'où la propagation automatique (section 5).

# 05

## Propager le contexte

Faire le pont entre ThreadLocal et Context, automatiquement.





# Micrometer Context Propagation

- La lib context-propagation définit des ThreadLocalAccessor : un pont entre un ThreadLocal et le Context Reactor.
- Les briques modernes (Micrometer Tracing, Spring Security réactif) enregistrent leurs accessors.
- Résultat : MDC, span, SecurityContext sont restaurés automatiquement à chaque changement de thread.

## L'IDÉE

Vous ne plongez plus chaque ThreadLocal à la main.  
La lib sait, pour chaque ThreadLocal enregistré, le sauver dans le Context et le restaurer à l'exécution.

# enableAutomaticContextPropagation

```
public static void main(String[] args) {
 Hooks.enableAutomaticContextPropagation();
 SpringApplication.run(PulseApp.class, args);
}
```

- Un seul appel au démarrage active la restauration automatique des ThreadLocal enregistrés.
- C'est la réponse 2026 : on garde MDC/SecurityContextHolder, le pont est transparent.
- Le contextWrite manuel reste utile pour vos propres clés (tenant, traceId métier).

# Un traceId de l'entrée au log SQL

- WebFilter en entrée : génère un traceId, le pose dans le MDC ET dans le Context.
- Propagation automatique : le traceId suit le pipeline malgré les changements de thread.
- Au moment de logger l'accès R2DBC, le traceId est présent — corrélation complète.

Prouver, logs à l'appui, qu'un même traceId apparaît de la requête HTTP jusqu'à la requête SQL, à travers les frontières async.

**OBJECTIF DU LAB Partie 3-1**

# 06

## Cache réactif & autres stores

Mettre en cache sans bloquer, et le cas Mongo.



## Mettre en cache un Mono

```
// cache explicite, réactif (reactor-extra / Caffeine)
CacheMono.lookup(key -> readFromCache(key),
 cacheKey)
 .onCacheMissResume(() -> directory.regionOf(id))
 .andWriteWith((k, sig) -> writeToCache(k, sig));
```

- Le @Cacheable classique ne déballe pas toujours bien les types réactifs : préférez un cache explicite.
- On cache la valeur, pas le Mono : sinon on rejoue un publisher déjà consommé.

# Repositories & tailable cursors

```
interface SampleRepo extends
 ReactiveMongoRepository<Sample, String> {

 @Tailable // flux infini sur capped collection
 Flux<Sample> findByAgent(String agent);
}
```

- MongoDB réactif : driver non bloquant natif, ReactiveMongoRepository.
- Les tailable cursors transforment une capped collection en Flux infini — idéal flux d'événements.

## Quel store réactif?

- SQL en flux, bout-en-bout exigé → R2DBC (en acceptant l'absence d'ORM).
- Domaine riche en relations, hors streaming → JDBC + Loom + JPA reste légitime.
- Flux d'événements append-only → Mongo tailable, ou Kafka (vu au prochain module).
- Cache : explicite et réactif, on met la valeur en cache, jamais le publisher.

# À vous : R2DBC & propagation de contexte

À faire à l'issue de ce module

- Persistance R2DBC (Testcontainers) + transaction réactive avec rollback prouvé.
- Un même `tracedId` du `WebFilter` jusqu'au log de la requête SQL.





## À RETENIR

# Données & Context — l'essentiel

- R2DBC = SQL réactif, non bloquant de bout en bout — mais pas d'ORM ; sinon, JDBC+Loom reste pertinent.
- Spring Data R2DBC : repositories Mono/Flux, mapping à plat, jointures explicites.
- Le backpressure remonte jusqu'au curseur SQL — l'argument fort du réactif sur les données.
- ThreadLocal casse aux frontières async : le Reactor Context (immuable, remonte la chaîne) le remplace.
- Propagation automatique (Micrometer + Hooks.enableAutomaticContextPropagation) : MDC/sécurité restaurés sans plomberie.



Cap sur les patterns — fan-out/fan-in, résilience, Kafka, observabilité et temps réel de bout en bout.

# Spring Reactor

Patterns & temps réel



résilience • idempotence • Kafka • backpressure bout-en-bout • observabilité

# Les six étapes du module

01

**Patterns de résilience**

fan-out • bulkhead • hedging

02

**Résilience : natif vs R4J**

circuit breaker • retry

03

**Idempotence & réessais**

at-least-once • dédup

04

**Messaging Kafka**

Spring Kafka bridgé

05

**Backpressure bout-en-bout**

client → DB / Kafka

06

**Observabilité**

Micrometer • tracing • OTel

**Fil rouge** — Pulse devient un système temps réel : les agents publient sur Kafka, le pipeline pousse en SSE, et tout est résilient et observable.

# 01

## Patterns de résilience

Composer des appels qui échouent, lentement ou pas du tout, sans tout faire tomber.



# La boîte à outils résilience

**timeout**

Borner l'attente — transformer la lenteur en erreur gérable.

**fallback**

Une réponse dégradée plutôt qu'une erreur (onErrorResume).

**retry**

Re-tenter, avec back-off et plafond (retryWhen).

**bulkhead**

Cloisonner : limiter la concurrence pour isoler les pannes.

**circuit breaker**

Couper vite quand un service est manifestement HS.

**hedging**

Doubler une requête lente, garder la première réponse.

# Survivre à un échec partiel

```
Flux.fromIterable(upstreams)
 .flatMap(u -> probe(u)
 .timeout(Duration.ofMillis(800))
 .onErrorReturn(Health.down(u)), 16) // borné
 .collectList();
```

- Chaque branche se protège seule : un upstream HS devient un résultat « down », pas un échec global.
- On agrège ce qui répond ; le timeout par branche évite qu'un lent bloque l'ensemble.

## PRINCIPE

Isoler la défaillance au niveau de la branche, décider de l'agrégat au niveau du fan-in.  
C'est l'agrégation de santé de Pulse — le cas où le réactif compose naturellement.

# Bulkhead & hedging

## BULKHEAD

Limiter la concurrence vers un service (flatMap borné, pool dédié).

Une panne sature sa cloison, pas tout le système.

## HEDGING

Si la réponse tarde, lancer une 2<sup>e</sup> requête et garder la 1<sup>re</sup> arrivée.

`Mono.firstWithSignal(a, b.delaySubscription(...))`.

## CAS DIFFÉRENTIEL — HEDGING

Le hedging réduit la latence de queue (p99) au prix de charge supplémentaire sur l'amont.

À réserver aux appels idempotents et aux SLA serrés — mesurer le surcoût.

# 02

## Résilience : natif vs Resilience4j

Spring Framework 7 en intègre une partie. Quand suffit-il, quand sortir l'artillerie ?





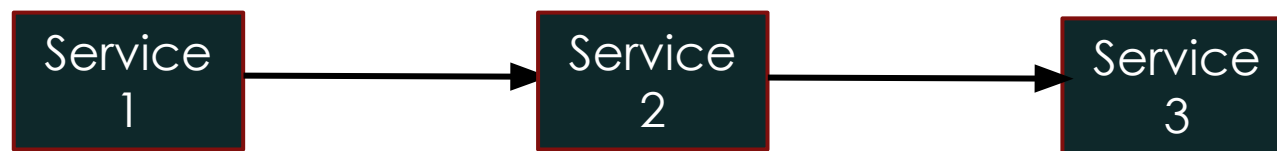
# Résilience

- Dans le cas où un service tombe ou n'atteint plus les performances attendues, c'est toute la chaîne qui est impactée
- Quand un service est appelé, il faut que ce dernier puisse notifier les services appelants qu'il a bien reçu la demande
- Après un certain délai sans réponse, l'appelant peut renvoyer sa demande ou alors trouver une alternative



# Resilience4J

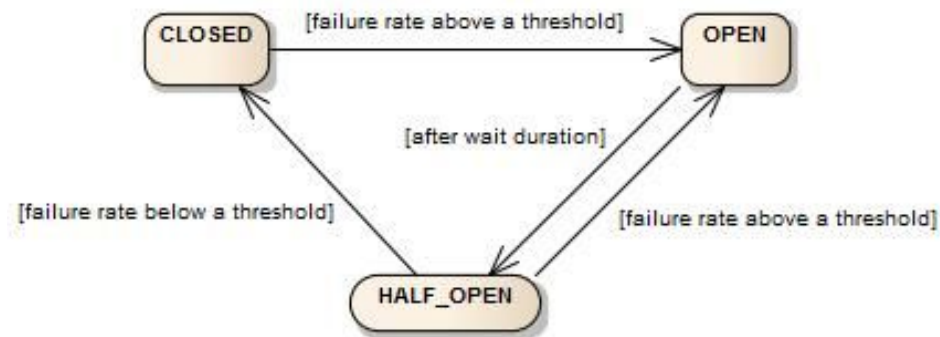
- Resilience4j est une bibliothèque légère de tolérance aux fautes conçue pour la programmation fonctionnelle.
- Elle fournit des fonctions d'ordre supérieur (décorateurs) pour améliorer n'importe quelle interface fonctionnelle, expression lambda ou référence de méthode avec un coupe-circuit, un limiteur de taux, une répétition ou une cloison.
- <https://resilience4j.readme.io/>



## Resilience4J – Annotations

|                        |                                                          |                                            |
|------------------------|----------------------------------------------------------|--------------------------------------------|
| <b>@Timeout</b>        | Limite la durée maximale d'exécution d'une méthode       | Lève une exception si le délai est dépassé |
| <b>@Retry</b>          | Réessaie automatiquement l'exécution en cas d'erreur     | Configurable : nombre de tentatives, délai |
| <b>@CircuitBreaker</b> | Coupe l'exécution après un certain nombre d'échecs       | Reprise après un délai configurable        |
| <b>@Fallback</b>       | Définit une méthode de secours (fallback) en cas d'échec | Appelée si les autres patterns échouent    |
| <b>@Bulkhead</b>       | Limite le nombre d'exécutions concurrentes               | Empêche une surcharge de ressources        |
| <b>@RateLimit</b>      | Limite le nombre d'exécutions sur une période de temps   | Utile pour les APIs externes/facturées     |

# Couper vite, se rétablir seul



- Au-delà d'un seuil d'échecs, le disjoncteur s'ouvre : on cesse d'appeler un service mort.
- Après une temporisation, il teste (semi-ouvert) ; si ça repasse, il referme.
- Effet : on échoue vite (fallback) au lieu d'empiler des appels voués à l'échec.

# Rate Limiter et Bulkhead

- **Rate Limiter**

- Il est possible de limiter le nombre d'appels sur une opération
- Il faut pour cela annoter l'opération avec l'annotation `@RateLimiter(name="mycall")`
- Ensuite, il faut ajouter les propriétés suivantes à l'application

```
resilience4j.ratelimiter.instances.mycall.limitForPeriod=2
resilience4j.ratelimiter.instances.mycall.limitRefreshPeriod=10s
```

- **Bulkhead**

- Il est possible de limiter le nombre d'appels concurrents
- Il faut pour cela annoter l'opération avec l'annotation `@Bulkhead(name="mycall")`
- Ensuite, il faut ajouter les propriétés suivantes à l'application

```
resilience4j.bulkhead.instances.mycall.maxConcurrentCalls=10
```

## Ce que Spring intègre désormais

```
@Retryable(maxAttempts = 3) // retour réactif OK
public Mono<Region> region(String id) { ... }
```

- Spring Framework 7 : @Retryable s'applique aux méthodes à retour réactif, throttling de concurrence intégré.
- Les Interface Clients (@HttpExchange) disposent d'un circuit breaker intégré côté Spring Cloud Commons.
- Pour beaucoup de cas (retry, CB simple sur appels sortants), le natif suffit — moins de dépendances.

## Quand il faut le chirurgical

```
// opérateurs réactifs Resilience4j
directory.regionOf(id)
 .transformDeferred(
 CircuitBreakerOperator.of(cb))
 .transformDeferred(
 BulkheadOperator.of(bulkhead));
```

- resilience4j-reactor fournit des opérateurs :  
CircuitBreaker, Bulkhead, RateLimiter,  
TimeLimiter.
- Composables via transformDeferred — fins  
réglages, métriques riches, registre central.

# Natif ou Resilience4j ?

## NATIF SF7 – PAR DÉFAUT

Retry, throttling, CB simple sur appels sortants.  
Zéro dépendance en plus.  
Le bon point de départ.

## RESILIENCE4J – SI BESOIN

Bulkhead, rate limiter, time limiter avancés.  
Réglages fins, métriques détaillées.  
Registre et configuration centralisés.



# 03

## Idempotence & réessais

Re-tenter sans danger suppose qu'on puisse rejouer sans dégât.



# At-least-once = doublons possibles

- Retry, Kafka, réseaux : la livraison est souvent at-least-once — le même message peut arriver deux fois.
- Un traitement non idempotent rejoué = double débit, double alerte, état corrompu.
- La parade n'est pas d'éviter les réessais, mais de rendre le traitement rejouable sans effet.

## LE PIÈGE

Mettre un `retryWhen` sur une opération non idempotente (ex. POST qui crée) = créer en double sur réessai.

# Clé d'idempotence & déduplication

```
// dédup par clé métier stable
Mono.from(repo.existsByEventId(e.id()))
 .flatMap(seen -> seen
 ? Mono.empty() // déjà traité
 : process(e));
```

- Une clé stable (eventId, idempotency-key) + une vérification = traitement au plus une fois effectif.
- Upsert ou contrainte d'unicité en base : la dédup peut aussi vivre côté stockage.

## Ce qui est rejouable, et le reste

- Lectures et opérations idempotentes (GET, PUT par clé, upsert) : rejouables sans risque.
- Créations / incréments : à protéger par clé d'idempotence avant d'autoriser le retry.
- `retryWhen(Retry.backoff)` avec filtre : ne re-tenter que les erreurs transitoires (timeout, 503), pas les 4xx.

```
Retry.backoff(3, Duration.ofMillis(200))
 .filter(ex -> isTransient(ex)) // pas les 4xx
```

# 04

## Messaging réactif : Kafka

Brancher un flux d'événements — avec le client qui existe vraiment en 2026.



# reactor-kafka n'est plus

- Reactor Kafka a été abandonné (mai 2025) et retiré du BOM Reactor 2025.0.
- Les templates réactifs de Spring for Apache Kafka sont dépréciés et voués à disparaître.
- La voie 2026 : Spring Kafka (client Java, bloquant par nature) bridgé vers un Flux.

## À NE PLUS UTILISER

reactor-kafka (EOL)

ReactiveKafkaConsumer/ProducerTemplate (dépréciés)

le binder Spring Cloud Stream Reactor Kafka (supprimé)

## Du @KafkaListener vers un Flux

```
Sinks.Many<Metric> sink = Sinks.many().multicast()
 .onBackpressureBuffer();

@KafkaListener(topics = "metrics")
void onMessage(Metric m) {
 sink.tryEmitNext(m); // pont vers le flux
}
```

- Le listener (bloquant) pousse dans un Sinks ; le reste du pipeline reste réactif.
- sink.asFlux() alimente l'agrégation et le SSE de Pulse.

## Publier depuis un pipeline

```
ingestion.normalize(raw)
 .flatMap(m -> Mono.fromFuture(
 kafkaTemplate.send("metrics", m)
 .toCompletableFuture()))
 .subscribe();
```

- `KafkaTemplate.send` renvoie un `CompletableFuture` : on l'enveloppe en `Mono.fromFuture`.
- On préserve l'ordre par partition via la clé ; `flatMap` borné pour ne pas noyer le broker.



# Backpressure côté Kafka

- Kafka est pull : le consumer poll() à son rythme — c'est un backpressure naturel par lot (max.poll.records).
- Le pont Sinks doit choisir sa stratégie d'overflow (buffer borné) si l'aval ralentit.
- Si l'aval bloque durablement, on met en pause la consommation (pause/resume) plutôt que de gonfler la mémoire.

## APARTÉ LOOM

Le client Kafka étant bloquant, un consumer sur virtual thread devient une option simple et viable.

Encore un cas où MVC+Loom mérite d'être pesé face au pont réactif.

# 05

## Backpressure de bout en bout

Du navigateur jusqu'à la base : où la pression remonte, où elle se rompt.



## Client → WebFlux → service → store

- Un client SSE lent demande moins ; la demande remonte le pipeline jusqu'à la lecture R2DBC ou au curseur.
- Tant que chaque maillon respecte request(n), la pression se propage sans buffer incontrôlé.
- C'est l'argument système du réactif : réguler la source depuis le consommateur, de bout en bout.

### PULSE DE BOUT EN BOUT

Curseur SQL / Kafka → pipeline → SSE → navigateur : un seul tuyau régulé, pas une suite de files qui débordent.

# Là où la chaîne se brise

- Sources hot non régulables (interval, événements externes) : pas de ralentissement possible → stratégie onBackpressure\*.
- Un pont (Sinks, callback, listener Kafka) interrompt la demande : il faut y décider buffer/drop/pause explicitement.
- Un opérateur qui matérialise (collectList sur flux infini, buffer non borné) casse le bénéfice.

## SYMPTÔME

Mémoire qui monte régulièrement = une file non bornée quelque part. Cherchez le maillon qui ne propage pas la demande.

# Réguler au niveau système

- Aux ponts : `onBackpressureBuffer` borné (+ alerte), ou pause/resume de la source pull (Kafka).
- `limitRate` / `prefetch` pour lisser la demande vers les amonts coûteux (base, API).
- Choix métier (rappel Partie 2) : une métrique périmée se jette (latest) ; un événement critique se met en file et s'alarme.

# 06

## Observabilité

Voir ce que fait un système réactif — métriques, traces, corrélation.



# Micrometer

```
Counter ingested = Counter.builder("pulse.ingested")
 .register(registry);

flux.doOnNext(m -> ingested.increment());
```

- Micrometer : façade de métriques, export Prometheus via `/actuator/prometheus`.
- Instrumentez aux points clés (taux d'ingestion, latence d'agrégation, ouvertures de circuit).
- Les opérateurs `doOn*` sont vos sondes — sans effet sur le flux nominal.

# Des spans à travers l'async

- Micrometer Tracing + l'API Observation produisent des spans corrélés par requête.
- Le tracing réactif repose sur la propagation de contexte vue en Partie 3-AM : sans elle, les spans cassent à chaque thread.
- Avec `Hooks.enableAutomaticContextPropagation()`, le span suit le pipeline — du `WebFilter` au call sortant.

## LE LIEN AVEC Partie 3

Observabilité et propagation de contexte sont les deux faces d'une même mécanique : le Context Reactor.



# Exporter vers la plateforme

- Micrometer Tracing se relie à OpenTelemetry (ou Brave) pour exporter vers Tempo, Jaeger, Zipkin...
- Métriques + traces + logs corrélés par le même traceId = investigation possible en prod.
- C'est le minimum pour opérer un système réactif : sans corrélation, le non bloquant est opaque.

# Tout corréler par le traceId

- Un incident Pulse : on part du traceId du log d'erreur, on retrouve le span, les métriques de la fenêtre, la requête SQL.
- La corrélation transforme un système réactif (par nature distribué dans le temps) en quelque chose de débogable.
- Investissez tôt : l'observabilité n'est pas une option pour le non bloquant, c'est une condition d'exploitation.

# À vous : Kafka, temps réel & résilience

À faire à l'issue de ce module

- Ingestion Kafka bridgée en Flux → SSE avec backpressure réel.
- Circuit breaker + observabilité, traceld corrélé de bout en bout.



# Patterns & temps réel — l'essentiel

- Résilience : isoler la défaillance par branche (fan-out), couper vite (circuit breaker), cloisonner (bulkhead).
- Natif SF7 d'abord (retry, CB sur appels sortants) ; Resilience4j pour le pointu (bulkhead, rate limiter).
- At-least-once → idempotence : clé stable + dédup avant d'autoriser les réessais.
- Kafka : reactor-kafka est mort — Spring Kafka bridgé en Flux (et Loom redevient une option).
- Backpressure bout-en-bout + observabilité (Micrometer/tracing/OTel corrélés par traceId) : conditions d'exploitation.



# Spring Reactor

Qualité, test & industrialisation



# Les six étapes du module

**01**

**La pyramide réactive**

tester des signaux

**02**

**StepVerifier avancé**

TestPublisher • Probe

**03**

**Tester le web**

WebTestClient • slices

**04**

**Testcontainers**

vraies bases en test

**05**

**BlockHound**

traquer le bloquant

**06**

**Industrialisation**

BOM • CI • versioning

**Fil rouge** — on durcit la suite de tests de Pulse, on garantit l'absence de blocage avec BlockHound, et on automatise tout en CI.

# 01

## La pyramide de test réactive

On ne teste pas une valeur de retour : on teste une séquence de signaux.



# Tester des signaux, pas des valeurs

- Un flux ne « retourne » rien : il émet `onNext*`, puis `onComplete` ou `onError`. On teste cette séquence.
- Outil de base (vu en Partie 1) : `StepVerifier` souscrit et vérifie le contrat, étape par étape.
- Le temps et la demande se simulent (virtual time, demande contrôlée) — pas d'attente réelle, pas de flakiness.

## CAS DIFFÉRENTIEL

Côté MVC : JUnit + `MockMvc` classiques, on asserte des valeurs.

Côté réactif : on asserte des signaux, et `BlockHound` traque le bloquant — un outil qui n'a pas d'équivalent en monde bloquant.



# Quel test pour quoi

## Unitaire (flux)

StepVerifier sur un pipeline pur — rapide, déterministe.

## Slice web

@WebFluxTest + WebTestClient — un contrôleur isolé.

## Intégration

Testcontainers : vraie base / Kafka, contexte complet.

## Correction non bloquante

BlockHound : aucun appel bloquant sur un pool non bloquant.

# Le socle StepVerifier

```
StepVerifier.create(pipeline.normalize(input))
 .assertNext(a -> assertThat(a.value())
 .isEqualTo(12.34))
 .expectNextCount(2)
 .expectComplete()
 .verify(Duration.ofSeconds(1));
```

- `assertNext` pour des assertions AssertJ riches ;  
`expectNext` pour l'égalité simple.
- `verify(Duration)` borne le test : un flux qui ne complète jamais échoue vite.

# 02

## StepVerifier avancé

Piloter la source, sonder les branches, dompter le temps.



## Piloter la source à la main

```
TestPublisher<Metric> tp = TestPublisher.create();
StepVerifier.create(pipeline.normalize(tp.flux()))
 .then(() -> tp.next(valid))
 .assertNext(...)
 .then(() -> tp.error(new IOException()))
 .expectError(IOException.class)
 .verify();
```

- TestPublisher émet à la demande : on contrôle précisément quand arrivent next/error/complete.
- createNoncompliant() permet même de tester la réaction à une source qui viole le contrat.

## Vérifier qu'une branche a été prise

```
PublisherProbe<Region> fb =
 PublisherProbe.of(Mono.just(UNKNOWN));

// le service utilise fb.mono() en fallback
StepVerifier.create(service.enrich(failing))
 .expectNextCount(1).verifyComplete();
fb.assertWasSubscribed(); // fallback pris
```

- PublisherProbe instrumente un Publisher :  
assertWasSubscribed / assertWasCancelled /  
assertWasRequested.
- Idéal pour prouver qu'un repli ou une branche  
d'erreur a (ou n'a pas) été emprunté.

# Tester les délais sans attendre

```
StepVerifier.withVirtualTime(() ->
 flux.delayElements(Duration.ofSeconds(1)))
 .thenAwait(Duration.ofSeconds(3))
 .expectNextCount(3)
 .verifyComplete();
```

- Rappel Partie 1 : le Supplier construit le flux APRÈS l'installation de l'horloge virtuelle.
- thenAwait avance le temps virtuel instantanément — back-off, intervals, timeouts testés en ms.

## Enregistrer & matcher

- `recordWith` + `consumeRecordedWith` : capturer tous les éléments pour une assertion globale (ordre indifférent).
- `expectNextMatches` / `expectErrorMatches` : prédicats quand l'égalité stricte ne suffit pas.
- `expectNoEvent(Duration)` : prouver qu'il ne se passe rien pendant un laps (cadence, debounce).
- `thenRequest` / `thenCancel` : piloter la demande pour tester le backpressure (vu en Partie 2).

# 03

## Tester le web

WebTestClient pour l'API réactive, slices pour isoler.





## Le client de test réactif

```
webTestClient.get().uri("/api/alerts")
 .exchange()
 .expectStatus().isOk()
 .expectBodyList(Alert.class)
 .hasSize(3);
```

- WebTestClient teste sans serveur réel (bindToController/Context) ou contre un serveur lancé.
- Pour un flux SSE :  
returnResult(...).getResponseBody() puis  
StepVerifier sur le corps.

## @WebFluxTest : un contrôleur isolé

```
@WebFluxTest(AlertController.class)
class AlertControllerTest {
 @Autowired WebTestClient client;
 @MockitoBean AlertService service; // mock
}
```

- @WebFluxTest ne charge que la couche web : rapide, ciblé sur le contrôleur.
- @MockitoBean (remplace @MockBean, déprécié) injecte un mock du service.

## Vérifier un endpoint streaming

```
Flux<MetricSample> body = client.get()
 .uri("/api/metrics/stream")
 .accept(MediaType.TEXT_EVENT_STREAM)
 .exchange().expectStatus().isOk()
 .returnResult(MetricSample.class)
 .getResponseBody();
StepVerifier.create(body.take(2))...
```

- On récupère le corps comme un Flux, puis on le pilote avec StepVerifier (take, virtual time).
- take(n) borne un flux infini pour que le test termine.

## Et côté MVC+Loom ?

- Le jumeau se teste classiquement : MockMvc, assertions sur des valeurs de retour.
- Mêmes contrats d'API → mêmes cas de test fonctionnels, deux styles d'assertion.
- C'est aussi un garde-fou : les deux implémentations doivent passer les mêmes tests de contrat.

### IDÉE

Un même jeu de tests de contrat (mêmes entrées/sorties) exécuté sur les deux apps verrouille l'équivalence fonctionnelle.

# 04

## Testcontainers

Tester sur de vraies bases, jetables et reproductibles.



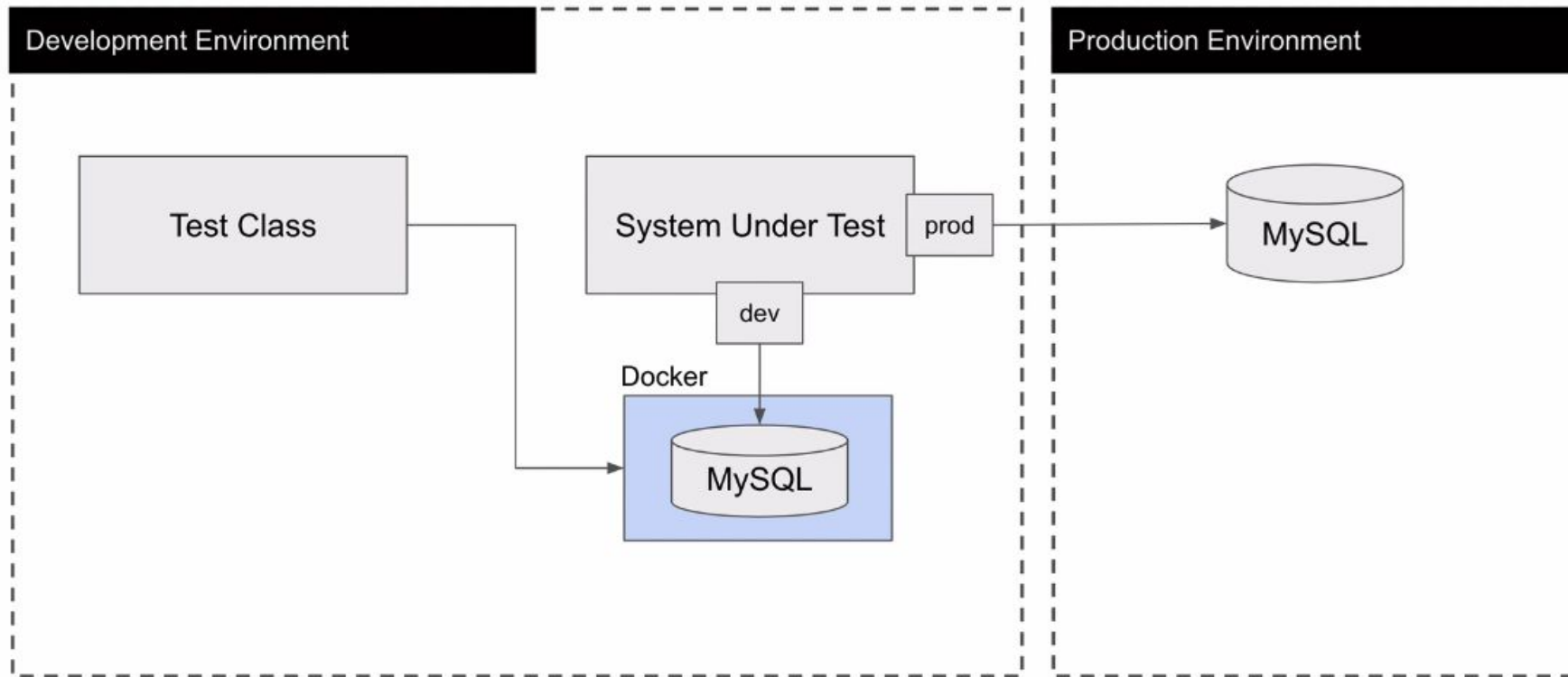
# Pas de mock pour l'intégration

- R2DBC, Kafka, Mongo ont des comportements (types, transactions, sémantique) qu'un mock ne reproduit pas.
- Testcontainers démarre un vrai PostgreSQL / Kafka en conteneur, le temps du test, puis le jette.
- Reproductible en local et en CI (Docker requis), sans base partagée fragile.

## À ÉVITER

Tester R2DBC contre H2 « pour aller vite » : les écarts de dialecte masquent ou inventent des bugs.

# Pas de mock pour l'intégration



<https://testcontainers.com/modules/?language=java>

# Câblage automatique

```
@Testcontainers
@SpringBootTest
class IngestionIT {
 @Container @ServiceConnection
 static PostgreSQLContainer<?> pg =
 new PostgreSQLContainer<>("postgres:17");
}
```

- @ServiceConnection configure tout seul spring.r2dbc.\* (et JDBC) depuis le conteneur.
- Plus de @DynamicPropertySource à la main pour l'URL/le port.



## Kafka → pipeline → SSE en test

- Un KafkaContainer + @ServiceConnection alimente le @KafkaListener réel de Pulse.
- On publie un message, on consomme le SSE via WebClientClient, on vérifie la propagation bout-en-bout.
- C'est le test qui prouve que tout le chemin temps réel tient, pas seulement chaque maillon.

### OBJECTIF LAB Partie 4-1

Un test d'intégration qui couvre Kafka → ingestion → SSE, avec le même traceId corrélé de bout en bout.

# 05

## BlockHound

L'agent qui interdit le bloquant là où il est interdit.



# Un appel bloquant invisible

- Un Thread.sleep, un JDBC, un fichier lu en bloquant sur un thread Netty/parallel : désastre sous charge.
- Le bug est invisible en test unitaire — il ne se voit qu'en prod, sous concurrence.
- BlockHound instrumente la JVM pour détecter tout appel bloquant sur un thread déclaré non bloquant.

## SANS BLOCKHOUND

On découvre le maillon bloquant en incident de production, au pire moment.

Avec : le test échoue immédiatement, en local, pointant la ligne fautive.

## En scope test

```
// dépendance test : blockhound-junit-platform
// s'active automatiquement pour la suite JUnit

@Test
void noBlockingCall() {
 StepVerifier.create(pipeline.run())
 .expectNextCount(10).verifyComplete();
} // échoue si un appel bloque un pool non bloquant
```

- Épingler une version de BlockHound compatible avec la JDK utilisée (instrumentation sensible à la version).

# Autoriser ce qui est isolé

- Certains blocages sont voulus et isolés sur boundedElastic (appel JDBC legacy enveloppé).
- On les déclare explicitement dans une allowlist BlockHound — un acte conscient, documenté.
- Tout le reste reste interdit : la liste blanche est l'exception, pas la règle.

## DISCIPLINE

Une entrée d'allowlist = une décision tracée. Si elle grossit sans raison, c'est un signal d'alerte sur la conception.

# Une garantie, pas un espoir

- BlockHound transforme « on pense que rien ne bloque » en « le build le prouve ».
- C'est le filet qui rend tenable le non bloquant de bout en bout à mesure que le code grossit.
- Côté jumeau MVC+Loom : aucun équivalent — bloquer y est normal. C'est un outil propre au réactif.

# 06

## Industrialisation

BOM, CI et versioning : rendre tout ça répétable.



# Le BOM gère, vous ne pinnez pas

- Le BOM Spring Boot 4 aligne Reactor 2025.0, Testcontainers, Micrometer, Jackson — versions cohérentes.
- On ne pinne pas reactor-core, reactor-test, etc. à la main : sources de conflits subtils.
- Une montée de version = bump du parent Boot, et on re-teste — pas un patchwork de versions.

## RÈGLE

Une seule source de vérité pour les versions : le BOM. Le reste du pom reste sans <version> sur les libs gérées.



# Ce que le pipeline doit garantir

**build**

Compilation + analyse statique.

**tests unitaires**

StepVerifier — rapides, sur chaque push.

**tests d'intégration**

Testcontainers (Docker dans la CI).

**BlockHound**

Aucun blocage sur pool non bloquant.

**gate**

Merge bloqué si l'un échoue.

# Versionner service et API

- Versionnez le service (semver) et l'API exposée : Spring Framework 7 intègre le versioning d'API REST.
- Une rupture de contrat = nouvelle version d'API, pas une modification silencieuse.
- Les tests de contrat (mêmes jeux réactif/jumeau) verrouillent la compatibilité au fil des versions.

# À vous : tests avancés, BlockHound & CI

À faire à l'issue de ce module

- TestPublisher / PublisherProbe, WebTestClient, Testcontainers @ServiceConnection.
- BlockHound actif + pipeline CI bloquant pour le merge.



## À RETENIR

# Qualité & industrialisation — l'essentiel

- On teste des signaux : StepVerifier, et ses outils avancés (TestPublisher, PublisherProbe, temps virtuel).
- WebClient + @WebFluxTest + @MockitoBean pour le web ; mêmes tests de contrat des deux côtés.
- Testcontainers (@ServiceConnection) pour de vraies bases/Kafka — jamais H2 à la place de R2DBC.
- BlockHound : le build prouve l'absence de blocage ; l'allowlist reste l'exception tracée.
- Le BOM gère les versions ; la CI gate build + tests + Testcontainers + BlockHound.



Cap sur la dernière ligne droite — perf, tuning, sécurité réactive, et la décision de migration sur chiffres.

# Spring Reactor

Perf, sécurité & décision



# Les six étapes du module

**01**

**Performance & tuning**

Netty • prefetch • GC

**02**

**Sécurité réactive**

authN/authZ non bloquantes

**03**

**Profiling**

JFR • ReactorDebugAgent

**04**

**Anti-patterns & revue**

checklist de PR

**05**

**Réactif ou Loom ?**

la décision, sur chiffres

**06**

**Runbook & exploitation**

SLO/SLI • canary

**Fil rouge** — on profile Pulse, on le sécurise, puis on charge le réactif et son jumeau MVC+Loom pour trancher la migration sur des mesures.

# 01

## Performance & tuning

Le réactif n'accélère pas une requête : il tient le débit. Ce qui se règle, et ce qui se surveille.



# Ce qui coûte vraiment

- Le réactif optimise le débit sous charge, pas la latence d'une requête isolée (rappel Partie 1).
- Peu de threads, mais des files internes : chaque opérateur a un prefetch (buffers en mémoire).
- Netty alloue des buffers directs (off-heap) : la mémoire ne se lit pas que dans le heap.
- Premier réflexe perf : mesurer (profiling, métriques), pas régler à l'aveugle.



# Les leviers serveur & client

```
HttpClient.create(
 ConnectionProvider.builder("pulse")
 .maxConnections(500)
 .pendingAcquireTimeout(ofSeconds(5)).build()
 .responseTimeout(Duration.ofSeconds(2));
```

- Boucle d'I/O : reactor.netty.ioWorkerCount (par défaut - nb de cœurs) — rarement à augmenter.
- Pool de connexions client : maxConnections, pendingAcquireTimeout — dimensionner selon l'amont.
- Timeouts response/read/write : toujours bornés, jamais infinis.

# prefetch, batch, limitRate

- Le prefetch (256 par défaut) = combien un opérateur demande d'avance. Plus haut = plus de débit, plus de mémoire.
- limitRate(n) lisse la demande vers un amont coûteux (base, API) sans tout tirer d'un coup.
- buffer(size/duration) pour les écritures par lots (DB, Kafka) — borner la taille, toujours.

## COMPROMIS

Gros prefetch / gros batch → débit en hausse, mémoire en hausse.

Petit → mémoire maîtrisée, plus d'aller-retours.

Se règle à la mesure, par étage du pipeline.

# Surveiller l'accumulation

- Moins de threads qu'en thread-per-request, mais des files d'opérateurs : une file non bornée = fuite lente.
- Symptôme classique : mémoire qui monte régulièrement → un pont (Sinks, buffer) ne propage pas la demande.
- GC moderne (G1, ZGC) ; surveiller le hors-tas (buffers directs Netty), pas seulement le heap.

## LE RÉFLEXE

Mémoire qui croît sans plateau ? Cherchez la file non bornée avant de toucher au GC — la cause est presque toujours là.

# 02

## Sécurité réactive

Authentification et autorisation non bloquantes — et le principal au fond du pipeline.



# SecurityWebFilterChain

```
@Bean
SecurityWebFilterChain security(
 ServerHttpSecurity http) {
 return http
 .authorizeExchange(e -> e
 .pathMatchers("/api/admin/**").hasRole("ADMIN")
 .anyExchange().authenticated())
 .oauth2ResourceServer(...).build();
}
```

- @EnableWebFluxSecurity + un SecurityWebFilterChain (DSL lambda) à la place du HttpSecurity servlet.
- authorizeExchange remplace authorizeRequests ; tout est non bloquant.

## Des composants réactifs

- ReactiveAuthenticationManager, ReactiveUserDetailsService : tout renvoie Mono — pas d'accès bloquant.
- JWT / OAuth2 resource server réactif : validation du token non bloquante.
- Un appel bloquant dans la sécurité (LDAP, JDBC) doit être isolé sur boundedElastic, comme ailleurs.

### PIÈGE

Un UserDetailsService bloquant branché tel quel sur la chaîne réactive gèle l'event-loop à chaque login.

# Lire l'utilisateur dans le pipeline

```
ReactiveSecurityContextHolder.getContext()
 .map(ctx -> ctx.getAuthentication().getName())
 .flatMap(user -> alertService
 .createFor(user, rule));
```

## LIEN AVEC Partie 3

Le SecurityContext est porté par le Reactor Context — il survit aux changements de thread grâce à la propagation.

- On ne lit plus SecurityContextHolder (ThreadLocal) : on lit le ReactiveSecurityContextHolder (Context).
- C'est exactement le mécanisme de propagation vu en Partie 3 — sécurité et contexte, même socle.

# 03

## Profiling & investigation

La pile d'exécution ne raconte pas le flux logique. Les bons outils, oui.





# Une stacktrace qui ment

- À l'exécution, la pile montre des threads d'event-loop et des internals Reactor, pas votre logique.
- Un goulot peut être un opérateur lent, un pool saturé, ou un blocage caché — invisible sans outillage.
- Deux familles d'outils : tracer l'assemblage (ReactorDebugAgent) et profiler l'exécution (JFR / async-profiler).

## PRINCIPE

On corrèle : la trace d'assemblage dit OÙ dans le code ; le profil dit SUR QUOI le temps part.

## ReactorDebugAgent (rappel Partie 2)

```
// au démarrage – coût négligeable, OK en prod
ReactorDebugAgent.init();

// ... une erreur produit une 'Assembly trace'
// pointant la ligne où l'opérateur a été assemblé
```

- L'agent instrumente le bytecode au démarrage : il relie l'erreur à votre code, pas aux internals.
- À garder actif en prod ; réserver `Hooks.onOperatorDebug()` (coûteux) au développement.

## JFR & async-profiler

- Java Flight Recorder : enregistre allocations, threads, I/O — overhead faible, utilisable en prod.
- Flame graphs (async-profiler) : voir où le temps CPU part, repérer un event-loop saturé ou un pool bloqué.
- Croisé avec BlockHound : un blocage révélé en test, un goulot révélé au profil.

```
java -XX:StartFlightRecording=duration=60s,\
 filename=pulse.jfr -jar pulse-reactive.jar
```

# 04

## Anti-patterns & revue

Les erreurs qui reviennent — et la checklist pour les attraper en PR.



# Les anti-patterns récurrents

**block() dans un pipeline**

Annule le non bloquant ; BlockHound doit l'attraper.

**subscribe() sauvage**

Dans un contrôleur : on retourne le Publisher, on ne souscrit pas.

**parallel() pour de l'I/O**

C'est pour le CPU ; l'I/O bloquant va sur boundedElastic.

**buffer non borné**

onBackpressureBuffer sans limite = fuite mémoire.

**bloquant sur l'event-loop**

JDBC/sleep sur un thread Netty gèle toutes ses requêtes.

# Le Mono qu'on oublie de retourner

```
// BUG : rien ne se passe, aucune erreur
public void save(Alert a) {
 repo.save(a); // jamais souscrit !
}

// CORRECT : on retourne le Mono
public Mono<Alert> save(Alert a) {
 return repo.save(a);
}
```

- Un Publisher non souscrit ne fait RIEN — pas d'erreur, juste une opération qui n'a pas lieu.
- Règle : on retourne toujours le Mono/Flux jusqu'au framework, qui souscrit.

# Checklist de PR réactive

- Aucun `block()/subscribe()` hors frontière ; tout `Publisher` est retourné ou composé.
- Tout `flatMap` vers de l'I/O a une concurrence bornée ; tout `buffer` a une taille.
- Timeouts présents sur les appels sortants ; erreurs traitées (`resume/retry` filtré).
- Pas de `ThreadLocal` nu pour le contexte transverse ; tests `StepVerifier` + `BlockHound` verts.

# 05

## Réactif ou Loom ?

La question du Partie 1, tranchée sur les chiffres du jumeau Pulse.





# On a mesuré, pas supposé

- Tout le cours a comparé pulse-reactive et pulse-mvc-loom à contrat identique.
- Sur le CRUD I/O-bound : débit comparable ; MVC+Loom plus simple à écrire et déboguer.
- Sur le streaming / forte concurrence I/O : le réactif gagne (backpressure, empreinte threads).

## LE MESSAGE

« Réactif partout » n'est pas la bonne réponse en 2026.  
« Le bon outil par charge », appuyé sur des mesures, l'est.

# Décider par facteur

|                                          |            |
|------------------------------------------|------------|
| Streaming / temps réel                   | Réactif    |
| CRUD I/O-bound simple                    | MVC + Loom |
| Backpressure bout-en-bout requis         | Réactif    |
| Domaine riche en relations (JPA)         | MVC + Loom |
| Forte concurrence I/O, empreinte threads | Réactif    |
| Équipe sans expérience réactive          | MVC + Loom |

# Migrer, ne pas migrer, hybrider

## NE PAS MIGRER

API CRUD sur JPA, charge modérée.  
MVC + Loom : plus simple, suffisant.

## MIGRER (CIBLÉ)

Ingestion / SSE / fan-out massif.  
WebFlux là où le flux le justifie.

## HYBRIDER

Gros CRUD + un module temps réel.  
Les deux modèles cohabitent.

*La migration n'est pas binaire : on migre ce qui le mérite, on garde le reste.*

## Faire cohabiter les deux

- Services séparés : un service WebFlux pour le temps réel, un service MVC+Loom pour le CRUD métier.
- Ou, à terme, virtual threads pour l'impératif et WebFlux pour les flux, selon l'endpoint.
- Critère : ne payer la complexité réactive que là où elle rapporte (mesuré), pas par principe.

### PULSE, AU FINAL

Le streaming d'ingestion reste réactif ; les règles d'alerte CRUD pourraient parfaitement vivre en MVC+Loom.

# 06

## Runbook & exploitation

Opérer un système réactif : objectifs, garde-fous, déploiement.



# Des objectifs mesurables

- SLI : latence p99, taux d'erreur, disponibilité — les indicateurs qu'on mesure réellement.
- SLO : la cible (ex. p99 < 200 ms, 99,9 % de succès) ; le budget d'erreur découle de l'écart.
- Le budget d'erreur arbitre : tant qu'il reste, on livre ; épuisé, on stabilise.

## RÉACTIF & SLO

Sous saturation, viser une dégradation douce (backpressure, fallback) plutôt que l'effondrement — c'est un choix de design, pas un hasard.

# Timeouts partout, résilience

- Un timeout sur chaque appel sortant et chaque accès données — jamais d'attente infinie.
- Circuit breaker + bulkhead sur les dépendances critiques (rappel Partie 3) ; fallback défini.
- Files bornées partout ; stratégie d'overflow explicite à chaque pont (Sinks, Kafka).

# Canary & feature flags

- Déploiement canari : router une fraction du trafic vers la nouvelle version, surveiller les SLI, élargir.
- Feature flags : activer un endpoint réactif progressivement, le couper sans redéployer en cas de souci.
- Particulièrement utile pour une migration ciblée : on bascule un flux à la fois, sous contrôle.



# Les signaux propres au réactif

- Saturation des pools (event-loop, boundedElastic), taille des files internes, demande non satisfaite.
- Ouvertures de circuit, taux de fallback, latences de queue (p99/p999) des appels sortants.
- Mémoire hors-tas (buffers Netty) et croissance des files — l'indicateur n°1 d'une fuite de backpressure.
- Le tout corrélé par traceId (Partie 3) : sans corrélation, le non bloquant reste opaque.

# À vous : sécurité, profiling & décision

À faire maintenant — capstone (jour 4)

- Sécurité réactive + profiling d'un goulot introduit puis corrigé.
- Charge réactif vs jumeau → décision + runbook (docs/decision-migration.md).



Modules pulse-reactive / -loadtest • ~3 h • solution : tag lab-j4-2

# Ce que vous emportez

- Maîtriser les flux : Mono/Flux, opérateurs, erreurs, threads et backpressure — testés sans attendre.
- Exposer du réactif : WebFlux, WebClient, R2DBC, SSE, le tout non bloquant de bout en bout.
- Le contexte propagé (trace, sécurité) et l'observabilité comme conditions d'exploitation.
- Résilience, idempotence, Kafka, et la qualité prouvée (BlockHound, Testcontainers, CI).
- Et surtout : décider réactif vs Loom sur des mesures — le bon outil par charge, pas par dogme.



*Merci — et bons flux.*